A project report on

CROSS-LINGUISTIC TRANSFER DIAGNOSTICS FOR LOW-RESOURCE FINANCIAL NLP IN INDIC LANGUAGES

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SOUPARNO NAG (23BCE5134)
ABHINAV RENJITH (23BCE1750)
KRISHNA SHARMA K (23BCE1763)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

OCTOBER 2026

A project report on

CROSS-LINGUISTIC TRANSFER DIAGNOSTICS FOR LOW-RESOURCE FINANCIAL NLP IN INDIC LANGUAGES

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SOUPARNO NAG (23BCE5134)
ABHINAV RENJITH (23BCE1750)
KRISHNA SHARMA K (23BCE1763)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

OCTOBER 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled “Cross-Linguistic Transfer Diagnostics for Low-Resource Financial NLP in Indic Languages” submitted by SOUPARNO NAG (23BCE5134), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of DR. MANIKANDAN P.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: Signature of the Candidate

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled “Cross-Linguistic Transfer Diagnostics for Low-Resource Financial NLP in Indic Languages” is prepared and submitted by SOUPARNO NAG (23BCE5134) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide: _____

Name: Dr. Manikandan P

Date: _____

Signature of the Examiner

_____

Name:

Date:

Signature of the Examiner

_____

Name:

Date:

Approved by the Head of Department,
B.Tech. Computer Science and Engineering

Name: DR Prassanna J

Date: _____

ABSTRACT

Multilingual language models do well on language–understanding tests yet it is unclear if they keep financial meaning intact when knowledge moves from one Indic language to another. The IndicFinNLP resource was made to give single-language baselines for Hindi, Bengali and Telugu. Its language splits differ in topic, entities and difficulty so any comparison between languages mixes the effect of language with the effect of content.

This project builds a controlled corpus where language's the only thing that changes. Each native split is translated directly between languages with IndicTrans2 without an English pivot into Hindi, Bengali, Telugu and Malayalam. The result is 92,760 machine-translated rows whose labels are never altered. Every row is checked for preservation, structural integrity and LaBSE similarity against a threshold set by human translations. Rows that fail a check are kept, never deleted. A pre-translation audit shows that the sustainability and ESG-theme splits already contain the items in every language while the numeral split is sourced separately.

IndicBERT and multilingual BERT were trained on a single language and then tested zero-shot on all other languages while keeping the text content unchanged. In sustainability classification the transfer gap is 0.26 to 0.45 macro-F1 for IndicBERT and 0.11 to 0.24 for mBERT. Hindi is the source language for both IndicBERT and mBERT; mBERT outperforms IndicBERT in every one of the twelve Hindi-source cases and shifting to a Dravidian language creates a larger drop than shifting to an Indo-Aryan language. I used a three-stage router to assign each of the 72 conditions a single cause. Linguistic causes explain 8.8 to 12.0 percent of the failures and two of the linguistic modules predict failure no better than random chance. Thus the report clearly shows the size and shape of the transfer gap. It leaves the reasons for that gap as an open question.

Keywords: cross-lingual transfer, financial NLP, Indic languages, machine translation, parallel corpus, IndicBERT, mBERT, failure diagnosis

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr Manikandan P, Assistant Professor Sr. Grade 1, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for constant guidance, continual encouragement and understanding; more than all, for teaching me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of NLP.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan, our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam, Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran, Vice-Chancellor, Dr. T. Thyagarajan, Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to DR Prassanna J, Head of the Department, B.Tech. Computer Science and Engineering and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: Souparno Nag

CONTENTS

LIST OF TABLES ..... **viii**

LIST OF FIGURES ..... **ix**

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE ..... **x**

CHAPTER 1 INTRODUCTION **1**

1.1 INTRODUCTION 1
1.2 OVERVIEW OF FINANCIAL NLP IN INDIC LANGUAGES 2
1.3 CHALLENGES IN CROSS-LINGUAL FINANCIAL NLP 3
1.4 PROBLEM STATEMENT 4
1.5 OBJECTIVES 4
1.6 SCOPE OF THE PROJECT 5
1.7 CONTRIBUTIONS 5

CHAPTER 2 BACKGROUND AND LITERATURE SURVEY **6**

2.1 INTRODUCTION 6
2.2 TECHNICAL BACKGROUND 6
2.3 FINANCIAL NLP RESOURCES AND MODELS 8
2.4 INDIC NLP RESOURCES AND BENCHMARKS 8
2.5 CROSS-LINGUAL TRANSFER 8
2.6 TRANSLATION-BASED TRANSFER AND ITS VALIDITY RISKS 9
2.7 COMPARISON OF THE REVIEWED STUDIES 9
2.8 GAP ANALYSIS AND POSITIONING 12
2.9 SUMMARY 12

CHAPTER 3 PROPOSED SYSTEM AND DESIGN **13**

3.1 SYSTEM OVERVIEW 13
3.2 DESIGN PRINCIPLES AND HARD RULES 14
3.3 SOURCE DATA 14
3.4 MODULE 1: INDEPENDENCE AUDIT 15
3.5 MODULE 2: ITEM ALIGNMENT 16
3.6 MODULE 3: IDENTIFIERS, LABELS AND STORAGE 17
3.7 MODULE 4: TRANSLATION 18
3.8 MODULE 5: TRANSLATION QUALITY GATES 19
3.9 MODULE 6: FREEZE AND DATASHEET 20
3.10 MODULE 7: EVALUATION-CONDITION MATRIX 21
3.11 MODULE 8: TRAINING AND ZERO-SHOT EVALUATION 21
3.12 MODULE 9: FAILURE DIAGNOSTIC ROUTER 23
3.13 MODULE 10: AGGREGATION AND FINDING RULE 25
3.14 DESIGN FOR LIMITED HARDWARE 26

CHAPTER 4 IMPLEMENTATION **27**

4.1 DEVELOPMENT ENVIRONMENT 27
4.2 REPOSITORY ORGANISATION 28
4.3 CORPUS CONSTRUCTION 29
4.4 TRANSLATION CORRUPTION MODES 31

4.5 THE QUALITY GATES 32
4.6 RESUME AND DETERMINISM PROBLEMS FOUND AND FIXED 33
4.7 TRAINING IMPLEMENTATION 34
4.8 DIAGNOSTIC IMPLEMENTATION 36
4.9 USER INTERFACE: RESULTS DASHBOARD 36
4.10 REPRODUCIBILITY MEASURES 37
CHAPTER 5 RESULTS AND DISCUSSION 38
5.1 INTRODUCTION 38
5.2 TESTING STRATEGY 38
5.3 EVALUATION METRICS 39
5.4 CORPUS QUALITY 40
5.5 IN-LANGUAGE BASELINES 44
5.6 ZERO-SHOT TRANSFER 45
5.7 ENCODER COMPARISON 47
5.8 SOURCE LANGUAGE AND DIRECTION 48
5.9 FAILURE ATTRIBUTION 49
5.10 VALIDATION 51
5.11 COMPARISON WITH PUBLISHED AND PRIOR RESULTS 52
5.12 DISCUSSION 53
5.13 LIMITATIONS AND THREATS TO VALIDITY 53
CHAPTER 6 CONCLUSION AND FUTURE WORK 55
6.1 CONCLUSION 55
6.2 FUTURE WORK 56
REFERENCES 58
APPENDIX 1 SCHEMA, LABELS AND IDENTIFIERS 63
A1.1 CORPUS ROW SCHEMA FOR TASK 1 63
A1.2 LABEL SETS 63
A1.3 IDENTIFIER RULES 64
APPENDIX 2 CONFIGURATIONS 65
A2.1 TRANSLATION AND VERIFICATION SETTINGS 65
A2.2 TRAINING AND DIAGNOSTIC SETTINGS 65
APPENDIX 3 PER-DIRECTION QUALITY TABLES 67
A3.1 ENTITY PRESERVATION 67
A3.2 SIMILARITY DISTRIBUTIONS 68
APPENDIX 4 FULL TRANSFER GAP TABLES 70
A4.1 TASK 2 AND TASK 3 TRANSFER GAPS 70
APPENDIX 5 REPRODUCTION AND SCRIPT INVENTORY 72
A5.1 SETTING UP 72
A5.2 COMMANDS 72
A5.3 PHASE AND TASK INVENTORY 73

LIST OF TABLES

Table 1.1: The four languages studied 2
Table 2.1: Comparison of reviewed studies, financial and Indic resources 10
Table 2.2: Comparison of reviewed studies, transfer and translation validity 11
Table 3.1: Hard rules of the corpus and the failure each one prevents 14
Table 3.2: The three IndicFinNLP tasks and their native row counts 15
Table 3.3: Canonical row schema 17
Table 4.1: Software and hardware environment 27
Table 4.2: Principal source modules 28
Table 4.3: Machine-translated rows generated per task 30
Table 4.4: Fine-tuning configuration 34
Table 4.5: Memory budget per encoder at maximum length 128 with fp16 35
Table 5.1: Automated tests by module 39
Table 5.2: Translation quality by task 40
Table 5.3: Survival of the annotated numeral in task 1 machine-translated rows 41
Table 5.4: Median LaBSE similarity and share below τ by direction 42
Table 5.5: Calibrated similarity threshold by task 43
Table 5.6: In-language macro-F1 on held-out test folds 44
Table 5.7: Macro-F1 by encoder, Hindi source 47
Table 5.8: Mean transfer gap by source language and target family, task 2 48
Table 5.9: Composition of transfer failures, share of failures 49
Table A1.1: Additional columns for the numeral task 63
Table A1.2: Task 3 classes and counts in each language 64
Table A2.1: Frozen translation decoding configuration 65
Table A2.2: Verification settings 65
Table A2.3: Diagnostic thresholds 66
Table A3.1: Entity preservation by direction, task 1 67
Table A3.2: Entity preservation by direction, task 2 67
Table A3.3: Entity preservation by direction, task 3 68
Table A3.4: LaBSE similarity distribution by direction, task 1 (τ = 0.82) 68
Table A3.5: LaBSE similarity distribution by direction, task 2 (τ = 0.82) 68
Table A3.6: LaBSE similarity distribution by direction, task 3 (τ = 0.73) 69
Table A4.1: Transfer gap by source, target and encoder, task 2 70
Table A4.2: Transfer gap by source, target and encoder, task 3 70
Table A5.1: Principal commands by phase 72
Table A5.2: Corpus tasks and what each delivers 73

LIST OF FIGURES

Figure 3.1: Overall architecture of the seven-phase pipeline 13
Figure 3.2: The three translation blocks and nine directions of each task 18
Figure 3.3: Corpus construction sequence and quality gates 19
Figure 3.4: Classifier architecture used for every source language and encoder 22
Figure 3.5: Three-stage diagnostic router 23
Figure 5.1: Outcome of the annotated numeral in task 1 machine translation 41
Figure 5.2: Entity preservation by direction and task, with the 95 percent bar 42
Figure 5.3: Median LaBSE similarity by translation direction and task 43
Figure 5.4: In-language macro-F1 for task 2 against the published figures 45
Figure 5.5: Task 2 transfer gap, source on the vertical axis and target on the horizontal axis 46
Figure 5.6: Task 3 transfer gap, source on the vertical axis and target on the horizontal axis 46
Figure 5.7: Mean transfer gap by source language on task 2 48
Figure 5.8: Composition of transfer failures by task and encoder 50

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

APA	American Psychological Association (reference style)
AUC	Area Under the (ROC) Curve
CC BY-NC-SA	Creative Commons Attribution-NonCommercial-ShareAlike licence
CI	Confidence Interval
CLS	Classification token of a BERT-style encoder
ESG	Environmental, Social and Governance
IG	Integrated Gradients
IndicBERT	ALBERT-based multilingual encoder pre-trained on Indian languages
IndicFinNLP	Financial NLP benchmark for Hindi, Bengali and Telugu
IndicTrans2	Machine-translation models for the 22 scheduled Indian languages
LaBSE	Language-agnostic BERT Sentence Embedding
mBERT	Multilingual BERT (cased, base)
ML	Machine Learning
MT	Machine Translation
NFC	Unicode Normalisation Form C
NLP	Natural Language Processing
OOM	Out of Memory
SHA-256	Secure Hash Algorithm, 256-bit
VRAM	Video Random Access Memory
XLM-R	Cross-lingual language model, RoBERTa-based
τ (tau)	LaBSE cosine-similarity threshold below which a translated pair is flagged as drift
cos(a, b)	Cosine similarity of two embedding vectors
F1	Harmonic mean of precision and recall
Gap(s→t)	Transfer gap: source in-language score minus target score
R_frag	Tokenizer fragmentation ratio (target tokens divided by source tokens)
hin, ben, tel, mal	Language codes for Hindi, Bengali, Telugu and Malayalam
hin_Deva, ben_Beng, tel_Telu, mal_Mlym	FLORES-style language codes used by IndicTrans2
H, B, T	Translation blocks whose native source is Hindi, Bengali and Telugu respectively

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Language technology for finance has far been built mostly for English. A bank, a regulator or a sustainability analyst in India however reads reports, news and filings in languages and the amount of financial text written in Indic languages is growing quickly. What is missing is labelled data [16]. Without labelled data, progress stalls. Annotating financial text is slow and needs domain knowledge so a typical Indic language has a few thousand labelled sentences, if any. A tempting shortcut is transfer: fine-tune a multilingual pre-trained encoder on the language that has labels and apply it without further training to a language that has none. Such zero-shot transfer works, on general benchmarks [7], [13], [28] but it degrades in low-resource settings and between languages that are far apart [19].

Finance makes the problem harder in ways that general benchmarks do not show. Financial sentences include a lot of numbers which have been written in Indian conventions like lakh and crore. This can come together with currency symbols and percentages, as well as named entities and technical terms which might not have a corresponding expression in the target language. When a model fails to differentiate between ten crore and ten million, it is not a tiny mistake. Rather it is an incorrect assumption about money.

This report presents a research project on how much of the financial classifier's competence remains when it is translated between Hindi, Bengali, Telugu and Malayalam, and why the rest is lost. This exercise is divided into seven phases. The first builds a parallel corpus in which the same content is available in all four languages. The second to fifth train classifiers and measure zero-shot transfer across languages and across two encoders. The sixth assigns each transfer failure a single cause. The seventh adds up those causes to see what the failures are made of.

1.2 OVERVIEW OF FINANCIAL NLP IN INDIC LANGUAGES

1.2.1 THE FOUR LANGUAGES

The study covers four languages that differ in family, script and morphology. The study looks at four languages that belong to families use different scripts and have different grammatical structures. Hindi and Bengali are both Indo-Aryan languages. Hindi uses the Devanagari script and Bengali uses the Bengali script. Telugu and Malayalam are Dravidian languages. They each have their unique scripts. Both Telugu and Malayalam are strongly agglutinative. This means that grammatical information is added to words through chains of suffixes. It is not expressed using function words. Table 1.1 gives a summary of the features that are important for this work.

Table 1.1: The four languages studied

Language	Family	Script	Digit range (Unicode)	Native data in IndicFinNLP
Hindi (hin)	Indo-Aryan	Devanagari	U+0966 to U+096F	Yes
Bengali (ben)	Indo-Aryan	Bengali	U+09E6 to U+09EF	Yes
Telugu (tel)	Dravidian	Telugu	U+0C66 to U+0C6F	Yes
Malayalam (mal)	Dravidian	Malayalam	U+0D66 to U+0D6F	No, machine translation only

Each script also has its own set of digits, and ASCII digits are used freely alongside them. A numeral may therefore appear as a Bengali digit string in one version of a sentence and as an ASCII string in another while denoting the same number. Any automatic check of whether a number survived translation has to compare values rather than characters.

1.2.2 FINANCIAL NLP RESOURCES

IndicFinNLP [12] brings natural language processing to Indian languages. It offers three tasks in Hindi, Bengali and Telugu. The tasks are identifying numerals in text, determining whether sentences are sustainable or unsustainable, and categorizing news headlines into one of ten ESG themes. The dataset was created to establish single-language baselines. The dataset was not created to test transfer learning. This focus shapes much of the design that follows.

1.2.3 WHY TRANSFER NEEDS A CONTROLLED CORPUS

If Hindi, Bengali and Telugu test sets have sentences about different companies, a lower score in Telugu compared to Hindi can be explained by the language itself but it can also be explained by the topic, the entity or the difficulty level. If we want to measure the transfer value cleanly, we must have the same content present in every language. Parallel test sets that have the content in every language exist for Indic natural language inference and for general understanding tasks [1], [9]. None of them possess ESG or finance specific material.

1.3 CHALLENGES IN CROSS-LINGUAL FINANCIAL NLP

The six challenges the project encountered were responsible for making a certain design decision in each case.

Absence of parallel financial data. IndicFinNLP does not make any claims about parallelism thus the corpus needs to be built. Machine translation is the only practical approach for building the corpus which comes with its own challenges.

Hidden parallelism. The split seems independent but it actually contains the content mixed. This will lead to evaluation of the model on the sentence from the training dataset written in a different script. The project takes care of ensuring independence before the translation.

Translation quality is the confound. Evaluation of machine-translated test set will lead to evaluation of the translator of the model and the low-resource languages suffer the most from this practice [2], [3]. The quality of the translation should be reported explicitly.

Numeric values (numbers) are fragile. Translation can drop a number, rewrite it in another scale, change its digit system or corrupt it. Text-level similarity scores cannot detect this.

No native Malayalam. IndicFinNLP has no Malayalam data so every Malayalam result mixes transfer difficulty with translation quality and no human reference exists to set a similarity threshold.

Limited compute. All experiments ran on a single laptop GPU with 4 GB of memory. Every long job had to checkpoint and resume and one of the three planned encoders could not be trained all.

A seventh methodological challenge runs through the project. A transfer failure can be explained in ways, and a plausible explanation is not evidence. Each diagnostic module therefore had to be validated against the data before its labels could be trusted.

1.4 PROBLEM STATEMENT

Do multilingual language models preserve financial meaning when knowledge moves between Indic languages? Answering this requires, first, a parallel test set in which language is the only variable, and whose translation quality is itself measured; and second, an analysis that explains transfer failures in terms of the language and the financial content (terminology, numerals, morphology and tokenisation) and not only in terms of overall accuracy.

1.5 OBJECTIVES

1. Audit the three IndicFinNLP language splits for each task to see whether the content is independent or shared before any translation is made.
2. Build a four-way parallel corpus in Hindi, Bengali, Telugu and Malayalam by translating from one Indic language to another keeping every label unchanged and never deleting any row.
3. Check every translated row for number preservation, structural integrity, and semantic similarity. Set the similarity threshold by comparing human translations wherever they exist.
4. Fine-tune two encoders on each language and measure zero-shot transfer across all combinations of source and target languages while keeping the content fixed after confirming the in-language baselines.
5. Compare source languages, target language families and encoders and report on which differences are larger than the seed noise.
6. Assign each transfer failure as a cause using an ordered diagnostic router, validate the modules, and report what the failures consist of, including the portion that no module can explain.

7. Release the corpus together with a datasheet that is generated from its own artefacts so that every figure in the documentation can be traced back to the data.

1.6 SCOPE OF THE PROJECT

The project includes three IndicFinNLP tasks, four languages, two encoders and nine translation directions. Corpus construction includes all three tasks. Transfer evaluation and diagnosis include the two classification tasks, the sustainability task (task 2) and the ESG-theme task (task 3). The numeral task (task 1) is not a classification task. It is used to check if numerals survive translation and it is the task whose language splits are separately collected.

The following are not included: training on machine-translated text, few-shot or adapter-based transfer, any language other than the four listed and an English pivot. XLM-R was supposed to be an encoder but could not be fine-tuned with the available GPU and the report treats that as a limitation not, as a result. The report does not claim to explain why transfer fails. It diagnoses the failure by observing it and making a record of explanations used to test it.

1.7 CONTRIBUTIONS

The work contributes the following.

A controlled four-way parallel financial corpus of 92,760 machine-translated rows, with an independence audit, an alignment of the shared items, quality gates on every row and a released datasheet.

A systematic zero-shot transfer study over three source languages and two encoders, with 126 planned results per task, all of which were completed.

The three-step diagnostic process where a semantic gate precedes the linguistic modules so that any interference caused by the translations produced in the process is not attributed to the model.

Negative results: two out of three linguistic modules fail to be able to predict failure better than randomly, tokenization fails to account for the encoder differences, and Hindi, the natural source language, is the worst choice.

CHAPTER 2
BACKGROUND AND LITERATURE SURVEY

2.1 INTRODUCTION

This chapter places the project in the literature. It first explains the technical building blocks the later chapters rely on, and then reviews twenty-eight peer-reviewed works in four groups: financial NLP resources, Indic NLP resources, cross-lingual transfer, and the validity of translation-based benchmarks. A comparison table and a gap analysis close the chapter.

2.2 TECHNICAL BACKGROUND

2.2.1 PRE-TRAINED MULTILINGUAL ENCODERS

An encoder such as BERT reads a sentence as a sequence of sub-word tokens and produces a vector for each token. A special classification token at the start of the sequence, written [CLS], is trained to summarise the whole sentence. For classification, a small linear layer is placed on top of this vector and the whole network is fine-tuned on labelled data. A multilingual encoder is pre-trained on text from many languages with a shared vocabulary, so that the same network can be fine-tuned in one language and applied in another. Multilingual BERT (mBERT) transfers across scripts but works best between languages of similar structure [26]. XLM-R improved results for languages with little data by scaling up pre-training, while exposing a trade-off between transfer quality and the capacity left for each language [7]. IndicBERT, released with IndicNLPSuite, is pre-trained on Indian languages only [17]. The tokenizer matters as well: how well a model's tokenizer handles a language strongly affects its performance on that language [29].

2.2.2 ZERO-SHOT TRANSFER AND THE TRANSFER GAP

In zero-shot transfer a model is fine-tuned on labelled data in a source language and then tested on a target language without any further training. The XTREME benchmark defines the transfer gap as the difference between a model's score in the source language and the model's score in the target language [13]. This project follows that

definition and calculates the transfer gap on identical items whenever the alignment permits.

2.2.3 MACHINE TRANSLATION FOR INDIAN LANGUAGES

IndicTrans2 provides translation models for all 22 scheduled Indian languages [11]. Its Indic-to-Indic checkpoints translate directly between two Indian languages without passing through English. A pivot through English would add a second source of error, and it would make it impossible to say which translation step introduced a given error. The project makes use of the 1 billion parameter version of the model.

2.2.4 LANGUAGE-AGNOSTIC SENTENCE EMBEDDINGS

LaBSE represents sentences in various languages in one vector space where translations of a sentence form clusters with the original sentence [10]. Cosine similarity between the vectors representing two sentences can be used to determine whether the sentences have similar meaning. The approach is employed in the project for two reasons: to verify if two groups of languages represent identical sentences and to evaluate the quality of each translated sentence against its source version.

2.2.5 EVALUATION MEASURES

The macro-F1 metric calculates the arithmetic mean of the F1 scores for all classes equally. This metric is mainly used in the current project because it does not allow the classifier to leverage the dominant class. Numeracy-600K demonstrated the differences in the results obtained with macro-F1 and micro-F1 in case of imbalanced classes [5], which is the case in the ESG-theme task.

2.2.6 ATTRIBUTION AND MORPHOLOGY

Two diagnostic methods are described in Chapter 3. Integrated Gradients is an attribution method which measures the influence of each input token on the prediction based on the accumulated gradients while traversing the path from the baseline input to the actual input. Morphological analyser is the tool used for stripping a word down to its roots. Telugu and Malayalam attach case markers and postpositions to the word, so a financial term can appear in a sentence only in an inflected form that a naive string match would miss. Integrated Gradients was introduced by Sundararajan et al. [31], and the morphological analyser is part of the Indic NLP Library [18].

2.3 FINANCIAL NLP RESOURCES AND MODELS

IndicFinNLP [12] is the resource this work builds on. IndicFinNLP provides nine datasets, three tasks in each of Hindi, Bengali and Telugu with baseline results. Outside India, MultiFin focuses on headline classification across 15 languages. It compares labelling by native speakers with a translate-then-label approach [15]. That comparison is a choice that every parallel dataset must make. Numeracy-600K treats magnitude as a learnable skill [5]. FinBERT shows that domain-specific pre-training beats a general BERT especially when training data are scarce [14]. FLUE and FLANG add an English financial benchmark and a pretrained model [30]. The ML-ESG shared tasks cover ESG impact classification in English, Chinese, Japanese and French, but include no Indic language [6]. A survey of financial language models lists low-resource evaluation, as a challenge [20].

2.4 INDIC NLP RESOURCES AND BENCHMARKS

IndicNLPSuite introduced IndicCorp and the first IndicBERT [17]. IndicXTREME extended this to nine tasks over 20 languages and introduced IndicBERT v2 [9]; many of its sub-tasks are parallel across languages, which sets a precedent for the corpus built here. IndicXNLI created a parallel inference set by machine translation, following the XNLI design [8], and compared transfer strategies, concluding that translate-train can outperform transfer from English for Indic languages [1]. IndoRE studied transfer among Bengali, Hindi and Telugu, the same triple that IndicFinNLP covers, for relation classification [24]. A study of part-of-speech tagging measured how much target-language data a few-shot transfer to Indic languages needs [27]. None of these resources includes ESG text.

2.5 CROSS-LINGUAL TRANSFER

Pires et al. [26] discovered that mBERT works between languages that are similar in structure and not so well between very different ones. Wu and Dredze [32] likewise found that mBERT transfers well zero-shot across many languages and tasks. XLM-R raised performance for lower-resource languages by scaling up [7]. XTREME tested how well models work across many languages and found that the gap is still large [13]. XTREME-R later revised the benchmark with more challenging tasks and settings [28]. Lauscher et al. [19] found that when a model tries to work with a language it has never

seen before, the results get worse if the language is very different from the source language or if there is not enough training data for the target language. However, even a small amount of data for the target language closes the gap significantly. Choosing the source language is a challenge on its own. It depends on how close two languages are how similar their vocabulary is, and how much data is available for each [22]. Adapter-based methods like MAD-X are an affordable way to adjust models than doing full fine-tuning [25]. Artetxe et al. [4] showed that transfer does not require a shared subword vocabulary or joint multilingual pre-training, and Libovický et al. [21] found that multilingual representations separate a language-specific component from a language-neutral one.

2.6 TRANSLATION-BASED TRANSFER AND ITS VALIDITY RISKS

Machine translation is used in parallel Indic benchmarks so the problems with machine translation are passed along. Artetxe et al. [3] found that translating the test set works better than earlier studies thought, once translation quality and the mismatch between training and test text are taken into consideration. Agrawal et al. [2] showed that translation errors in XNLI [8] hurt low-resource languages the most and that manually re-annotating Hindi and Urdu did not match the labels taken from English. A systematic review of multilingual sentiment analysis concluded that creating resources for low resource languages from high-resource ones is still a methodologically open question [23].

2.7 COMPARISON OF THE REVIEWED STUDIES

Tables 2.1 and 2.2 compare the twenty-eight studies on the property that matters for this work. The "Relation to this project" column states what the project takes from each study. The comparison draws only on what each study claims; it does not rank them.

Table 2.1: Comparison of reviewed studies, financial and Indic resources

Study	Year	Focus and contribution	Languages / domain	Relation to this project
Ghosh et al. [12]	2024	IndicFinNLP: nine datasets, three financial tasks, baselines	Hindi, Bengali, Telugu; finance	Source data for all three tasks; baselines used for validation
Jørgensen et al. [15]	2023	MultiFin: headline classification; native labelling versus	15 languages; finance	Motivates stating how labels were obtained in a parallel set

Study	Year	Focus and contribution	Languages / domain	Relation to this project
		translate-then-label		
Chen et al. [5]	2019	Numeracy-600K: magnitude as a learnable skill; macro-F1 versus micro-F1	English; finance	Supports numeral checks and macro-F1 as the primary metric
Huang et al. [14]	2023	FinBERT: domain pre-training beats general BERT with limited data	English; finance	Context for domain-specific encoders
Chen et al. [6]	2023	ML-ESG-2: ESG impact type identification	English, Chinese, Japanese, French	Shows no Indic ESG shared task existed
Lee et al. [20]	2025	Survey of financial language models	Multiple; finance	Lists low-resource evaluation as open
Kakwani et al. [17]	2020	IndicNLPSuite: IndicCorp and first IndicBERT	Indian languages; general	Source of the IndicBERT checkpoint
Doddapaneni et al. [9]	2023	IndicXTREME and IndicBERT v2: nine tasks, 20 languages	Indian languages; general	Precedent for n-way parallel Indic evaluation
Aggarwal et al. [1]	2022	IndicXNLI: MT-built parallel inference set; transfer strategies	Indian languages; NLI	Precedent for MT-built parallel Indic benchmarks
Nag et al. [24]	2023	IndoRE: transfer among Bengali, Hindi, Telugu	Same triple; relation classification	Same language triple as IndicFinNLP
Raj & Kumar [27]	2026	Data needed for few-shot Indic POS transfer	Indic languages; POS tagging	Few-shot budget, listed as future work
Shah et al. [30]	2022	FLUE benchmark and FLANG model	English; finance	Shows existing financial benchmarks are English-only
Joshi et al. [16]	2020	Taxonomy of language resource gaps	Many languages; general	Frames Indic languages as under-resourced

Table 2.2: Comparison of reviewed studies, transfer and translation validity

Study	Year	Focus and contribution	Languages / domain	Relation to this project
Pires et al. [26]	2019	mBERT transfers across scripts, best between similar languages	Multiple; general	mBERT is one of the two encoders
Conneau et al. [7]	2020	XLM-R: scale improves low-resource transfer, with a capacity trade-off	Many languages; general	Planned third encoder; capacity question left open
Hu et al. [13]	2020	XTREME: transfer gap across tasks and languages	Many languages; general	Definition of the transfer gap used here
Lauscher et al. [19]	2020	Zero-shot transfer worsens for distant languages; few target examples help	Multiple; general	Motivates the Dravidian-target analysis

Study	Year	Focus and contribution	Languages / domain	Relation to this project
Lin et al. [22]	2019	Choosing transfer languages from relatedness and data size	Multiple; general	Motivates the source-language comparison
Pfeiffer et al. [25]	2020	MAD-X: adapter-based multi-task transfer	Multiple; general	Adapters evaluated and rejected as confound
Artetxe et al. [3]	2023	Translate-test revisited with quality and mismatch controls	Multiple; classification	Translation quality must be controlled
Agrawal et al. [2]	2024	Translation errors in XNLI hurt low-resource languages	Hindi, Urdu and others; NLI	Warns that MT-built labels may be invalid
Mabokela et al. [23]	2023	Review of multilingual sentiment analysis	Under-resourced languages	Methodological openness of resource creation
Gala et al. [11]	2023	IndicTrans2: MT for 22 scheduled Indian languages	22 Indian languages	Translation model used for the corpus
Feng et al. [10]	2022	LaBSE: language-agnostic sentence embeddings	Many languages	Similarity model for audit and quality gate
Wu & Dredze [32]	2019	mBERT zero-shot transfer across tasks	Many languages; general	Supports using mBERT as a source-language encoder
Conneau et al. [8]	2018	XNLI translated evaluation set	15 languages; NLI	Origin of the translated-benchmark approach
Artetxe et al. [4]	2020	Transfer without shared vocabulary	Multiple; general	Informs the capacity-versus-breadth question
Libovický et al. [21]	2020	Language-specific and language-neutral components	Multiple; general	Context for language-identity effects
Rust et al. [29]	2021	Tokenizer quality and per-language performance	Many languages; general	Motivates the fragmentation module
Ruder et al. [28]	2021	XTREME-R, a revised benchmark	Many languages; general	Companion to the transfer-gap definition

2.8 GAP ANALYSIS AND POSITIONING

All the individual methods are present. The missing part is how they are combined. Parallel Indic evaluation for general understanding tasks exist [1], [9] but not for finance, and multilingual ESG work has not covered Indic languages [6]. Transfer gaps are usually reported in aggregate [13], or explained by general language similarity [26], [19], [22]. They are rarely linked, item by item to specific content features such as

numbers, terminology or morphology. Few-shot budgets and source-language choice have not been studied for financial text. Ultimately, if we require evidence that machine-translated benchmarks that are unverified will actually measure translation quality instead of transfer [2]. Therefore, any newly created parallel corpus needs to have clear measures of quality. This project tackles these gaps at once: it builds the parallel financial corpus, evaluates its own translation quality and links each transfer failure to a documented cause.

2.9 SUMMARY

The literature supports three working assumptions. Multilingual encoders transfer imperfectly and unevenly, so a measured gap is expected. Machine-translated benchmarks are only as trustworthy as their quality checks, so the checks must be built in. And the literature offers no Indic financial parallel benchmark, so one has to be constructed before the transfer question can be asked.

CHAPTER 3

PROPOSED SYSTEM AND DESIGN

3.1 SYSTEM OVERVIEW

The system is a pipeline that takes the three native IndicFinNLP splits and turns them into a four-language corpus. Then it tests classifiers on that corpus. Each time a transfer fails the system records why it failed. Figure 3.1 shows the seven phases and what each one passes on to the next. A phase never goes back to an earlier one unless it uses a frozen artefact. That's what lets later results be traced all the way back to the data that produced them.

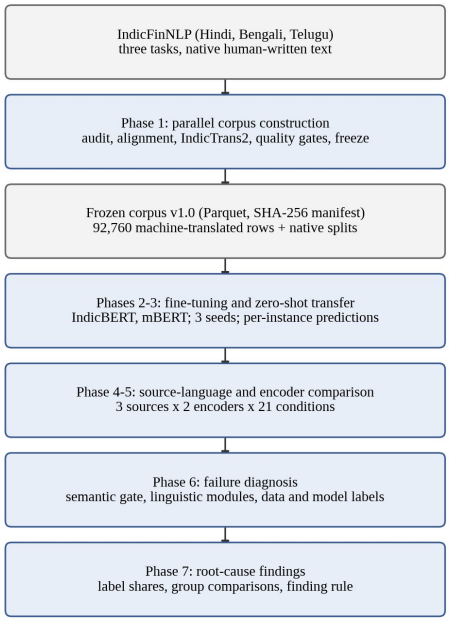


Figure 3.1: Overall architecture of the seven-phase pipeline

Three decisions apply to every phase. First, there will never be any row deleted from the corpus. The flagging system records low similarity score, blank output, or corrupted data to make sure that failure is kept in the measure rather than filtered out. Second, every script receives an explicitly specified random seed, ensuring byte-for-byte reproducibility of output for the same configuration. Third, every artefact records a hash of the configuration that produced it.

3.2 DESIGN PRINCIPLES AND HARD RULES

This design follows certain number of principles that are determined beforehand. Each of them protects from a certain type of silent error. Table 3.1 lists them with the failure they prevent.

Table 3.1: Hard rules of the corpus and the failure each one prevents

Rule	Failure it prevents
Never drop a row; flag it	Silent row loss that would corrupt every downstream result
Parquet only, never CSV for corpus data	Indic scripts and numerals breaking under CSV quoting
No pickle for corpus data	Unreadable or unsafe released artefacts
No English pivot	A second error source that makes drift attribution meaningless
No modification of the frozen release	Results that cannot be traced to fixed data
Labels come from one schema and are never remapped	Label drift between source and translated rows
Explicit seed on every script	Runs that cannot be reproduced
Configuration hash logged with every artefact	Artefacts that cannot be tied to the settings that made them
No Unicode normalisation of task 1 text without recomputing offsets	Silent corruption of character offsets

The last rule needs a word of explanation. Task 1 marks a number by character offsets into the raw string. Unicode normalisation to the NFC form decomposes precomposed nukta letters and composes Telugu vowel signs, so the text length changes while the text still looks correct. Normalising the upstream task 1 text would break 880 of its 22,786 annotated spans, and nothing visible would reveal it. For this reason normalisation is treated as corruption for task 1 and not as cleaning.

3.3 SOURCE DATA

The corpus starts from IndicFinNLP [12]. The upstream files are kept byte for byte, with SHA-256 checksums recorded in a manifest, and are read once at the ingest boundary. Table 3.2 lists the three tasks.

Table 3.2: The three IndicFinNLP tasks and their native row counts

Task	Content	Type	Hindi	Bengali	Telugu
Task 1	Numerals in financial text	Span annotation	10,640	6,130	6,016
Task 2	Sustainability sentences	Binary	2,238	2,228	2,072

Task	Content	Type	Hindi	Bengali	Telugu
		classification			
Task 3	ESG news headlines	10-class classification	532	532	532

For Task 1, there is no class label. It consists of a number identified in the sentence through a beginning and ending offset, a scale mark, and the number itself in the source script and in ASCII digits. Therefore, one task 1 instance is a sentence and span and not merely a sentence. This means that a sentence containing three numbers will appear three times. Tasks 2 and 3 are classic classification datasets. The two classes for task 2 are “sustainable” and “unsustainable” and for task 3 ten ESG themes.

3.4 MODULE 1: INDEPENDENCE AUDIT

3.4.1 PURPOSE

The comparison design relies on having three independently created native splits. This module checks to see if this requirement is true before any translation takes place. If the splits are truly just the same sentences in rearranged, then testing the model on a different language will involve testing it on its own training data.

3.4.2 TESTS

An analysis based only on the comparison of randomly selected cross-language pairs is meaningless, as randomly selected pairs from any two corpora will get a poor score regardless of the parallel nature of the two corpora. This audit uses three tests which are performed using LaBSE embeddings.

B3, structural. Comparison is made of the number of rows and class distribution. If the same classes occur in two data sets, it can be implied that the data sets originated from the same source. In case there exists an identifier like a URL which links the two data sets together, then this test is sufficient.

B1, order-aligned. Cosine similarity for each pair of rows (i.e., row i of source and row i of target) is compared with a row shuffle control group.

B2, nearest neighbour. For each of 200 sentences sampled in the source, the maximum cosine similarity across the whole target split is calculated. This test

continues to fire even when the rows were permuted or deleted in part, which means the finding stands.

Since an obvious nearest neighbour metric may not be easily interpreted within one narrow context, the audit also includes calibration controls. Positive control is the data split that is known to be parallel. The negative control is the text which is not parallel but comes from the same region and language pair. Only if the negative controls lie well below the threshold τ will the finding stand.

Algorithm 3.1: Independence test for two language splits

Input: source split S, target split R, sample size n = 200, threshold τ, seed
1. Embed every sentence of S and R with LaBSE; L2-normalise the vectors
2. Draw n sentences from S using the seed
3. For each drawn sentence s, compute m(s) = max over r in R of cos(s, r)
4. Report the median of m and the share of m(s) at or above τ
5. Repeat on the positive control and on each negative control
6. Declare the pair parallel only if the share at or above τ is far above the negative controls

3.5 MODULE 2: ITEM ALIGNMENT

This audit splits the work in two parts, and module two deals with the latter. Task 3 has the URL field, and the alignment here will be exact match on this key and does not require any model. Task 2 does not have a key, and so alignment will use mutual nearest neighbors on LaBSE similarity above τ, and an item must have agreement in all three language pairs. Mutual matching avoids ten source sentences to be aligned to just one target sentence, and three way agreement ensures that this alignment was made via two separate ways. Algorithm 3.2 provides the approach.

Algorithm 3.2: Three-way alignment for task 2

Input: Hindi, Bengali and Telugu splits, threshold τ
1. For each language pair (a, b), compute the cosine matrix between the splits
2. Keep a pair (i, j) only if j is the nearest neighbour of i, i is the nearest neighbour of j, and cos(i, j) >= τ
3. Compose the pairwise maps: Hindi to Bengali to Telugu, and Hindi to Telugu directly
4. Keep an item only if both routes agree on every language
5. Assign a content-derived item identifier to the aligned item
6. Keep every unaligned row in its own split with an unaligned flag
7. Fail the run if any aligned item carries conflicting labels

Misaligned rows are flagged and retained. A conflicting label constitutes a severe error because the corpus cannot contain a gold standard label for any item that exhibits discrepancies between its languages. Task 1 was obtained separately, and therefore no alignment procedure was performed on it, as it did not have any alignment to begin with.

3.6 MODULE 3: IDENTIFIERS, LABELS AND STORAGE

Every corpus Parquet file has the same columns, with the task carried by the file path. Table 3.3 lists them. The pair of block identifier and item identifier, together with the language, forms the primary key.

Table 3.3: Canonical row schema

Column	Type	Meaning
block_id	string	H, B or T: the block whose native split the item comes from
item_id	string	Content-derived key; the join key across languages
lang	string	hin, ben, tel or mal
origin	string	native or mt
src_lang	string	Source language of a translation; null for native rows
text	string	The sentence
label, label_id	string, integer	Gold label and its canonical index (tasks 2 and 3)
labse_sim	float	Cosine similarity with the source; null for native rows
flags	list of strings	Quality flags such as translation_drift or entity_loss

The item identifier is derived from content, never from row position. A positional identifier would silently point at a different item if the upstream files were reordered. Because a task 1 item is a sentence and span pair, its identifier includes the span. Exact duplicate rows receive an occurrence suffix, so that the four exact duplicates in task 2 stay distinct and none is dropped. Task 1 uses the same columns with the label columns replaced by the number as written, the number in ASCII digits, the start and end offsets, the scale marker and a flag that records whether the span was recovered after translation.

All reads and writes pass through one input and output module, which enforces the format, the schema and the label set. A loader raises an error on any label outside the schema and on any label identifier that disagrees with it; it never coerces or skips.

3.7 MODULE 4: TRANSLATION

3.7.1 BLOCK DESIGN

Each task is built as three blocks, one per native language. Within a block the native split is translated into the other three languages, which gives nine translation directions and twelve splits per task (Figure 3.2). All splits within a block share item identifiers and gold labels. IndicFinNLP has no Malayalam, so Malayalam exists only as machine translation, once per block. This is a documented limitation and no attempt is made to synthesise a native version.

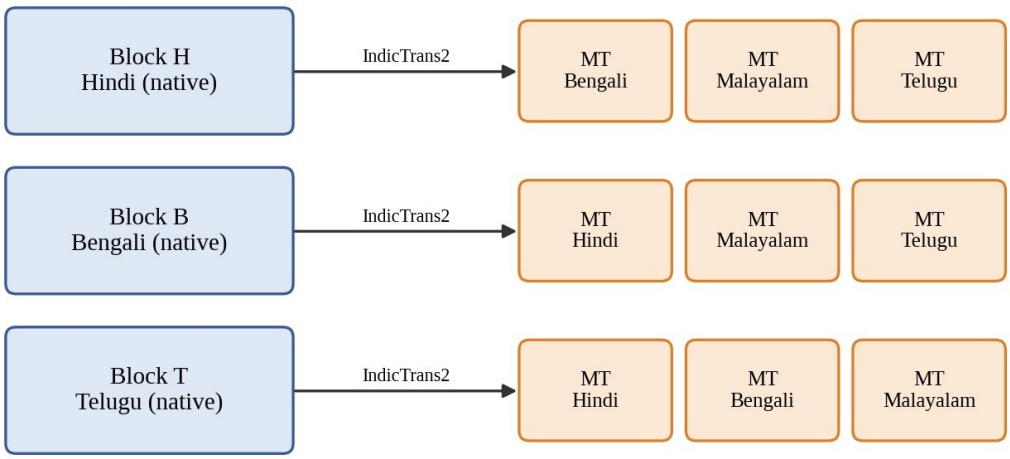


Figure 3.2: The three translation blocks and nine directions of each task

3.7.2 TRANSLATION MODEL AND DECODING

Translation uses the 1B-parameter Indic-to-Indic IndicTrans2 checkpoint directly, with no English pivot. One decoding configuration is applied to all nine directions, so that drift figures are comparable: five beams, a maximum length of 256 tokens, no sampling. A decoding fingerprint, a hash of everything that can change the output text, is logged with every artefact. Batch size is deliberately left out of the fingerprint because it must not affect the output, and any dependence on it would be a defect worth catching.

The 1B model was preferred over the 320M distilled variant on measured evidence. On a 180-row sample of task 1 the smaller model was eight times faster but corrupted 10.0 percent of rows, against 4.4 percent for the larger one. A digit-preservation score had suggested the opposite, because it asks whether numbers survived and not whether the sentence came out intact.

Identical source sentences are translated once and the result is shared. In task 1 a sentence with three numbers forms three items. Translating each separately would cost up to three times as much and could yield three different translations of one sentence, differing only in which number was annotated.

3.8 MODULE 5: TRANSLATION QUALITY GATES

There are three automatic tests performed on each translated row. Each of these tests failure modes which the other two tests cannot detect, as illustrated in Figure 3.3.

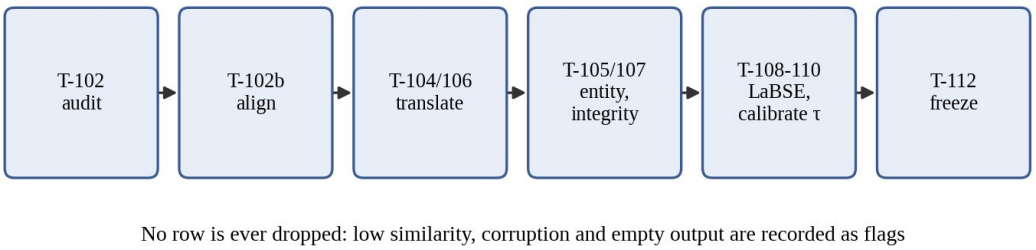


Figure 3.3: Corpus construction sequence and quality gates

3.8.1 NUMERAL AND ENTITY PRESERVATION

Number, currency, percentage and lakh/crore are compared in terms of value across systems. There were two things that needed to be done. One of them was scales: one hundred million and ten crore are both ten to the power eight. So normalization occurs by transforming the number into its scaled value. Second thousands separators: a comma is a separator only when three digits follow it so that a date such as 22, 2019 reads as two numbers and not as 22,2019. Those rows that have a corruption of the type mentioned in Chapter 4 are left out of the calculation of the preservation rate so that it cannot be said that there is a loss due to the processing of numerals. A direction whose preservation rate is than 95 percent is noted.

3.8.2 STRUCTURAL INTEGRITY

Checks for blocking need every block to attach four-way on the item identifier all translated rows to have the labels of their corresponding source row and the native text to have no replacement characters. If any of these conditions fails the whole process will stop. Checks for non-blocking detect script leakage meaning the appearance of

characters from some Indic script except for the shared danda punctuation mark the two corruption mechanisms noticed in the output of IndicTrans2, namely the presence of unconverted entity placeholders and nukta characters that appear as literal escape sequences and truncation. Truncation is determined separately per direction as half of the direction's median length ratio since the Malayalam translation is ten percent longer, than its Hindi source while the Bengali one is four percent shorter.

3.8.3 SEMANTIC SIMILARITY

Source and translation are embedded with LaBSE and compared by cosine similarity over all 92,760 pairs, with no sampling, defined here in Equation 3.1:

$$\mathrm{sim}(x, y) = (e(x) \cdot e(y)) / (\|e(x)\| \, \|e(y)\|) \tag{3.1}$$

where $e(\cdot)$ is the LaBSE embedding. A pair with similarity below the threshold τ receives the flag translation_drift and stays in the corpus. For tasks 2 and 3 the native split in the target language is a human translation of the same item, and a human translation is a known-good pair. The threshold can therefore be calibrated: it should accept nearly all human pairs. The calibrated values are reported in Chapter 5. For task 1, and for every Malayalam target, no human reference exists, and the report says so wherever it applies.

3.9 MODULE 6: FREEZE AND DATASHEET

When a task passes its checks, its twelve splits are copied into a release directory, hashed file by file into a manifest, and made read-only. The manifest records what produced the bytes as well as the bytes themselves: the translation model, the decoding fingerprint, the similarity model and the threshold. A verification function fails if a file is modified, deleted or added. The release datasheet is generated from the frozen manifests and the reports and is never typed by hand, so that it cannot drift from the data. A check mode fails if the committed datasheet no longer matches the artefacts.

3.10 MODULE 7: EVALUATION-CONDITION MATRIX

The splits which may act as training source, evaluation target or neither is fixed in one configuration file before any model is trained. For tasks 2 and 3, the evaluation matrix consists of 21 conditions. Nine of these represent cross-lingual transfer, covering all combinations of the three source languages with the other two native languages and

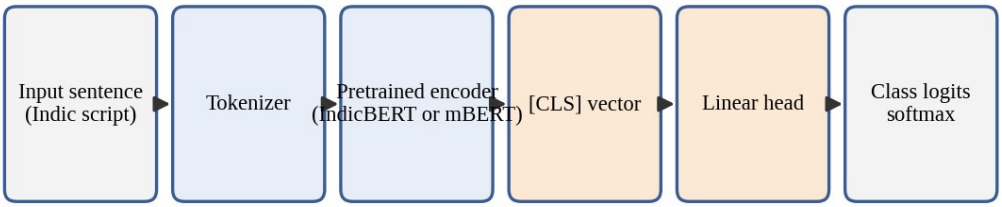
Malayalam. Another nine repeat these transfer experiments using machine-translated target text from the corresponding source-language block. The remaining three examine translationese by keeping the language constant while changing the text's origin. For Hindi these include native text, text translated from Bengali and text translated from Telugu. The transfer cells can be classified into four typological quadrants: Indo-Aryan to Indo-Aryan, Indo-Aryan to Dravidian, Dravidian to Indo-Aryan and Dravidian to Dravidian.

Two protections are enforced. At split level, no split is both a training source and an evaluation target within a single condition. At item level, the training and evaluation item sets are disjoint, and the items are partitioned by a seeded hash of the item identifier and not by position. Hashing makes the partition a property of the item, so every language agrees on it without coordination. Evaluation prefers native text, so that a failure reflects transfer and not translation. Malayalam has no native text, so its evaluation data is always machine-translated, taken from a block other than the one trained on, and its provenance is recorded with each result.

3.11 MODULE 8: TRAINING AND ZERO-SHOT EVALUATION

3.11.1 MODEL

Each classifier is a pre-trained encoder with a single linear classification head on the [CLS] representation, trained with cross-entropy loss (Figure 3.4). The design is deliberately minimal, so that any difference between runs can be attributed to the encoder or the language and not to the head. Two encoders are compared, IndicBERT [17] and cased multilingual BERT [26]. XLM-R [7] was planned as a third, and it is discussed in Chapter 5.



Cross-entropy loss; AdamW; fp16; early stopping on development macro-F1

Figure 3.4: Classifier architecture used for every source language and encoder

3.11.2 TRAINING RULES

Training uses native written text by humans only, so that no transfer measurement is contaminated by the project's own translations. Every configuration is run with three seeds and reported as mean and standard deviation. Every source model is first checked against IndicFinNLPs published in-language macro-F1 with a tolerance of 0.02. A shortfall must be identified before any transfer result is considered. This makes sure that a big difference cannot be the result of a poorly trained source model.

3.11.3 ZERO-SHOT EVALUATION

Evaluation is done without any training and is completely zero-shot. The model checkpoints are loaded in a read-only format. During inference the system checks that no optimizer is present and no gradient updates happen. Each prediction is logged with the item, the label, the predicted label and the class probabilities. Macro-F1 is the main metric. Accuracy and F1 scores are also reported for each class. Confidence intervals at 95 percent are calculated using a bootstrap method with 1,000 samples over the items. The transfer gap, from source s to target t is measured following the XTREME method [13]:

$$\text{Gap}(s \rightarrow t) = \text{F1}(s \text{ on } s \text{ items}) - \text{F1}(s \text{ on } t \text{ items}) \tag{3.2}$$

computed on the same underlying items wherever the alignment allows. A gap smaller than its own seed standard deviation is not reported as a finding.

3.12 MODULE 9: FAILURE DIAGNOSTIC ROUTER

A transfer failure is an item that the model classified correctly in the source language and incorrectly in the target language, as presented in Equation 3.3:

$$Fail(i) = 1[\ \hat{y}_src(i) = y(i)\ ] \cdot 1[\ \hat{y}_tgt(i) \neq y(i)\ ] \qquad (3.3)$$

Predictions are connected based on the combination of item and seed. So a prediction from the source is never matched with a prediction from the target that comes from a different seed. Since the model answered this item correctly in the source language the possibility of not having the right skills and the difficulty of the item are excluded by the design itself.

Each failure receives exactly one cause label from an ordered router, with an audit trail of every module that fired (Figure 3.5).

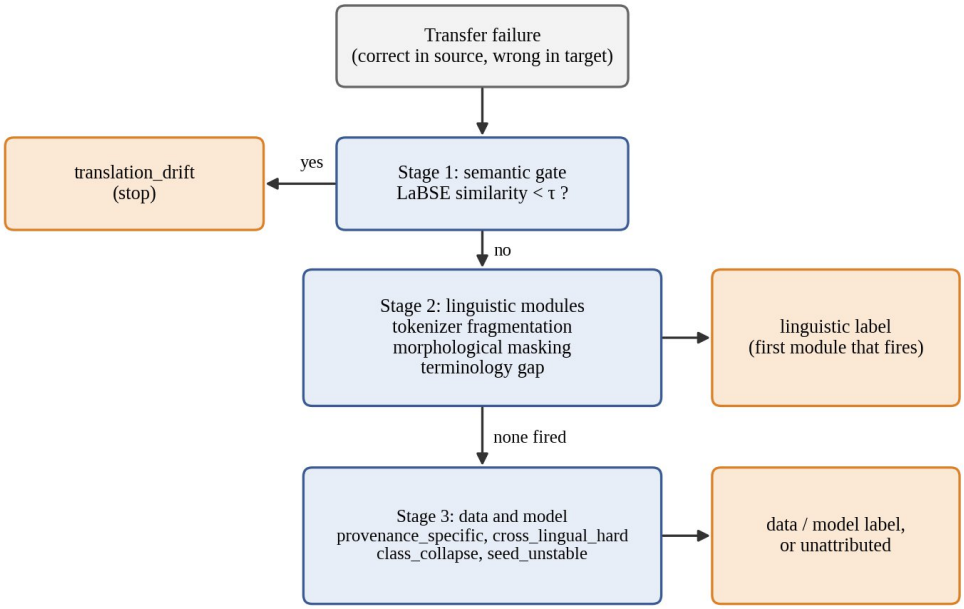


Figure 3.5: Three-stage diagnostic router

3.12.1 STAGE 1: SEMANTIC GATE

The failure is labelled translation_drift and the process stops if the LaBSE similarity between the source text and the target text is lower than τ. The sequence is important. If the linguistic modules run first the noise that comes from the projects translations might be blamed on tokenisation or morphology. The gate reuses the corpus similarity module exactly as it is without making any changes.

3.12.2 STAGE 2: LINGUISTIC MODULES

The remaining failures are tested against three modules about the target text.

Tokenizer fragmentation measures how much more finely the encoder's own tokenizer splits the target text than the source text, a factor known to affect per-language performance [29], as defined in Equation 3.4::

R_frag = (number of target tokens) / (number of source tokens) (3.4)

and fires when R_frag is 2.0 or more.

Morphological masking fires when a financial term from an 18-concept ESG lexicon appears in the target in an inflected form only, whose root is hidden because of morphology. The Indic NLP Library's morphological analyser provides the stems, with rule-based suffix stripping where no analyser exists. A stem matches only exactly, with its final code point dropped, or with one added vowel sign.

Terminology gap fires when Integrated Gradients attributions which computed against a zero-embedding baseline diverge between the source item and its target counterpart. The threshold is set per task and encoder at the 95th percentile of divergence on pairs the model classified correctly in both languages. An attribution whose completeness error exceeds 5 percent is marked unconverged and never fires.

A fourth module, on orthographic and numeral mismatch, was planned and dropped. A digit audit showed that items whose digit system differs between native and machine-translated text fail at or below the base rate, so a label from that module would have been spurious.

3.12.3 STAGE 3: DATA AND MODEL BEHAVIOUR

Stage 3 claims only what the first two stages leave unattributed. It describes a failure by the item or the model and not by the target sentence. Four labels are used:

provenance_specific: the same item and seed is correct on the twin target of the other provenance;

cross_lingual_hard: the item fails in all three targets of the same source;

class_collapse: the run predicts one class for at least 75 percent of the condition's rows, and the failure is that class;

seed_unstable: another seed classified the item correctly.

The precedence is fixed as translation_drift, tokenizer_fragmentation, morphological_masking, terminology_gap, provenance_specific, cross_lingual_hard, class_collapse, seed_unstable and unattributed. Each label belongs to one of four groups: linguistic, data, model or unattributed. The unattributed share is reported as an outcome and is not hidden.

Algorithm 3.3: Diagnostic router for one transfer failure

Input: failed item i with seed k, source text, target text, encoder, threshold τ
1. If cos(source, target) < τ, return translation_drift
2. Compute R_frag with the encoder tokenizer; if R_frag >= 2.0, record tokenizer_fragmentation
3. Test morphological masking against the ESG lexicon and record it if it fires
4. Compute Integrated Gradients divergence; if converged and above the cell threshold, record terminology_gap
5. If any Stage 2 module fired, return the first in precedence order
6. Test provenance_specific, cross_lingual_hard, class_collapse, seed_unstable in that order
7. If none fired, return unattributed
8. Always store the audit trail of every module that fired

3.13 MODULE 10: AGGREGATION AND FINDING RULE

The unit of analysis is a failed (item, seed) row, and a group's share of a label is the fraction of that group's failures that carry it. Shares therefore describe what failures consist of. Labels are not compared against correct predictions, so a share does not show that a property makes failure more likely. Each share carries a 95 percent bootstrap interval that resamples (condition, item) clusters, so an item that fails on all three seeds counts once. Groups are compared by target direction, source language, encoder and text provenance.

A difference counts as a finding only if the two intervals do not overlap and the difference points the same way on both encoders (on both tasks, for the encoder comparison). This rule was fixed before any result table was read. Every comparison is repeated without the source runs that never trained. A difference that a known measurement limit could produce by itself is reported with that caveat attached. For example, Hindi targets can never receive the morphological label, because Hindi writes postpositions as separate words.

3.14 DESIGN FOR LIMITED HARDWARE

All experiments ran on one laptop GPU with 4 GB of memory, and the design reflects it. Every GPU job checkpoints after each batch and resumes by item key. Translation batches are halved and retried on out-of-memory errors. A batch that exceeds a time limit is treated like an out-of-memory error, so one stalled batch cannot consume a night's run. Jobs refuse to fall back silently to the CPU, because CPU arithmetic in 32-bit floats yields different translations from the GPU's 16-bit arithmetic, and a corpus generated half on each would not be comparable while looking normal.

CHAPTER 4
IMPLEMENTATION

4.1 DEVELOPMENT ENVIRONMENT

The project is written in Python 3.11, and every entry point asserts the interpreter version and fails loudly on a mismatch. It runs on Linux or WSL2, because the IndicTrans2 toolkit is not built or tested for Windows. Table 4.1 lists the main components and the reason for each choice.

Table 4.1: Software and hardware environment

Component	Choice	Reason
Language	Python 3.11	Required by the toolkit; asserted at entry
Translation	ai4bharat/indictrans2-indic-indic-1B	Direct Indic-to-Indic, no English pivot
Similarity	sentence-transformers/LaBSE	Language-agnostic sentence embeddings
Encoders	ai4bharat/indic-bert, bert-base-multilingual-cased	The two encoders compared
Libraries	transformers 4.57.6, sentence-transformers 5.7.0	transformers must stay below version 5
Storage	Parquet (pyarrow), JSON configs	Safe for Indic scripts and numerals
GPU	NVIDIA RTX 3050 Laptop, 4 GB	The only device available
Dashboard	Streamlit	Read-only exploration of results

4.1.1 THE TRANSFORMERS VERSION PIN

IndicTrans2 checkpoints ship with their modelling code. This code is loaded with remote-code trust. Is built using the 4.x series of the transformers library. Version 5 breaks it in three ways: first, a removed ONNX module that is imported by the configuration file; second a moved tokenizer base class that is imported by the toolkit; and third, a tokenizer that assigns an attribute before initializing its parent class. The first two issues can be worked around with shims. The third one cannot be fixed without modifying private internals of a class that comes from remote code. That kind of change would not be reproducible. So the library is locked to a version, below 5. We checked

that this downgrade does not affect the similarity model's output. After downgrading, the task 2 alignment reproduced bit for bit.

The checkpoint's modelling code checks the first element of the past key values at each decoding step. In modern versions, a cache object is passed and this cache object starts off empty at the first step. Because of this, the cache must be turned off in the decoding configuration. This reduces decoding speed. It is required for correctness and is not something that can be adjusted for performance.

4.2 REPOSITORY ORGANISATION

The corpus modules are kept flat in one source directory, with one thin command-line script per task, named by task identifier. Table 4.2 lists the modules and what each does.

Table 4.2: Principal source modules

Module	Responsibility
corpus_io	Every read and write of corpus data; format, schema and label enforcement
ids	Content-derived key construction, block mapping, join helpers
audit, align	Independence audit and item alignment
translate	IndicTrans2 wrapper with resumable, adaptive batching
spans	Re-location of task 1 numerals in translated text
entities	Scale-aware numeral, currency and percentage comparison
generate	Native split plus translations to canonical machine-translated rows
integrity	Structural, leakage and corruption checks
labse_gate	Embedding, cosine similarity, thresholding
freeze, datasheet	Hashing, manifest, immutability, generated datasheet
conditions	Evaluation-condition matrix and validator
data, config, train	Encoder registry, hashed YAML configuration, fine-tuning loop
inference, predictions, metrics	Read-only evaluation, per-instance logs, macro-F1 and bootstrap
transfer, mismatch, failures	Transfer gap, failure extraction
diagnostics, fragmentation, morphology, saliency, behaviour	The router and its modules
findings	Phase 7 aggregation and the finding rule

No module other than the input and output module reads or writes corpus Parquet files directly. This ensures that the format and label rules are enforced in one place. It also prevents those rules from being copied into every script. Everything runs as a module, from the repository root. This keeps path resolution anchored to the root.

4.3 CORPUS CONSTRUCTION

4.3.1 INGEST AND THE INPUT/OUTPUT LAYER

The upstream spreadsheets are read once. Then they are converted to Parquet format. The loader rejects any label that is not recognized. It also rejects a label identifier upon not matching the expected schema. A round-trip test is run to write and read a fixture. This fixture includes Malayalam text, Telugu digits, a currency symbol and a lakh or crore expression. The result is compared bytewise to ensure the lack of data loss. Unaligned rows are ingested with an identifier and a flag indicating they are unaligned. This keeps them in the corpus but outside the parallel set. The native splits contain 22,786, 6,538 and 1,596 rows for the three tasks. None of the rows were dropped.

4.3.2 TRANSLATION WRAPPER

The translation module adds the model to the toolkits before and after processing steps. The model is loaded once and used again for different directions because loading a 4.8 GB model nine times is not efficient, and on a 4 GB card, it caused a memory problem the second time it was loaded. After every direction, the allocator is cleared, because the data is removed but the blocks are not returned so each direction would start with less memory than the one before.

The card cannot handle a fixed batch size. In Task 1, the Telugu split contains 280 rows that are longer than 1,000 characters while the median length is only 117. Therefore a batch that is sized for an average row will fail when it meets one of those rows. Beam search keeps an array that contains beam times batch times vocabulary logits as 32-bit floats. That array allocation fails first. The adaptive translation routine therefore cuts the batch size in half after an out-of-memory error. It tries again, shrinking all the way down to a single row. Only if one row cannot fit will it raise an error. The order of the rows stays the same. The memory allocator is configured to use segments before the deep learning library is imported so that beam search does not fail because of fragmentation, instead only failing when real memory is exhausted.

Every run writes a checkpoint after each batch, keyed by item identifier, so an interrupted run resumes instead of restarting. A failed batch is flagged and the run continues; an empty output is flagged and never dropped.

4.3.3 GENERATION OF THE NINE DIRECTIONS

The generation module takes a split along with its translations and converts them into canonical rows. It makes sure the number of rows matches the source exactly. It then checks that item identifiers and labels are the same, across all entries. It also adds a flag to every empty translation. If any of these checks fail it raises an error of just reporting the issue. Tasks were generated in order of increasing cost, starting with task 3. This way the full path was tested end to end in under two hours before any GPU time was used for task 1. Table 4.3 shows the volumes.

Table 4.3: Machine-translated rows generated per task

Task	Rows to translate	Distinct sentences	Approx. GPU time
Task 1	68,358	40,401	About 14 hours
Task 2	19,614	19,602	Estimated 3 to 7 hours
Task 3	4,788	4,788	37 minutes

4.3.4 SPAN RECOVERY AND ENTITY COMPARISON

Span module matches a number by its value not by its text. A Bengali digit string, a string and a grouped number such as 25,000 are all found and 22, 2019 is read as two separate numbers. A list of numbers loses the spaces that separated them when translated, so 145, 146, 150 can become 145,146,150. The comma is then read again as a separator. Only when the quantity did not move: in 50 thousand rewritten as 50, 000, the digits belong to the rescaled number and a span on the leading 50 would point at the wrong characters while claiming success. This rule on the span module recovered 255 rows.

The entity module compares numerals, currency and percentages after normalising both sides to a value with its scale applied. We validated the entity module against output rather than only against fixtures. The first version of the entity module reported 68 to 94 percent preservation. When we inspected the flagged rows of the entity module, we found three bugs in the checker and none in the model: a space after the comma that caused the entity module to parse 50, 000 as 50 and 0; a corruption mode

that transliterated into the target script and matched two of four script forms; and the entity module compared currency as a literal symbol so the rupee sign against its Telugu form was read as a loss. After we applied fixes to the entity module, the altered counts fell from 6 to 8 per direction, to 0 to 1 on the test sample. The test fixtures include seven pairs that must compare equal because a checker that reports loss that has not occurred is the failure to avoid.

4.4 TRANSLATION CORRUPTION MODES

Two silent corruption modes in IndicTrans2 output were found during implementation. Both destroy content while leaving fluent text, so neither is visible without an explicit check.

The first issue is called entity placeholder leakage where the pre-processor replaces entities with placeholders like an identifier tag and restores them back after decoding. The model sometimes translates the placeholder text itself, for example, into the Bengali or Malayalam version of the letters ID. When it tries to match the real content via restoration, it doesn't match. This can also cause numeric ranges to disappear. The second issue is called escape leakage. A nukta character, like the Bengali letter with code point U+09BC can appear as the escape string u09bc when changed into the target script. In the sample, all eight rows that were affected had a nukta in the source and no row without one had a leak. Across the corpus 607 of task 1's 734 escape leaks come from Bengali source with 106 from Hindi source and 21 from Telugu source, so the problem is mostly from Bengali but not only Bengali.

The first estimate of the corruption rate was one percent. That number turned out to be wrong. The reason was that the detector used covered two of the four target scripts. When we used a corrected detector, the rate came out to 3.2 percent. This was based on 900 rows of translation smoke samples. Every target language showed signs of corruption. This is why we now validate the checkers against real output.

4.5 THE QUALITY GATES

4.5.1 STRUCTURAL INTEGRITY

The integrity module divides its checks into two levels. Blocking checks deal with a block that stops joining four ways, a translated label that moved away from where it

came from, and a replacement character in text. These issues mean that the corpus makes a false statement about a row which causes the run to fail. Findings include leakage, corruption and possible truncation. They look at the translation and not the corpus so their rows are kept, and they do not cause the run to fail unless a strict mode is turned on. The module can add its flags to the corpus in a way that does not change the result if done more than once. Its own flags are removed before being created, so doing it twice is the same as doing it once. This was checked by running it and looking at every flag list.

4.5.2 SIMILARITY SCORING

Embeddings are L2-normalised when they are encoded. Because of this, cosine similarity reduces to a dot product. No downstream process has to remember to normalise. Embeddings are cached for each split with keys that include the row count and the file path. When a split is regenerated with a number of rows, it must not silently reuse vectors that belong to other sentences. Doing so would attach every score to the pair even though it might look fine. A second similarity is computed for tasks 2 and 3: this similarity compares the translation to the native human translation in the same language. That comparison forms the basis of the calibration in Chapter 5.

4.5.3 DRIFT FLAGS, RANKING AND FREEZE

Below-threshold rows get the translation_drift flag and stay in the corpus; the flag is created again each time so that changing τ does not leave old flags. The directions are sorted by the average of each measures rank and not by a sum that has weights because a drift rate and a numeral-preservation rate do not use the scale, and making up weights would create a false sense of accuracy that the data does not have. The freeze step saves the per-task τ in each manifest. While making the datasheet, we discovered that the global τ had been placed in every manifest so task 3's manifest had 0.82 while its drift flags had been created using 0.73. This was fixed and Task 3 was frozen again.

4.6 RESUME AND DETERMINISM PROBLEMS FOUND AND FIXED

Four defects in the long-running jobs deserve record because each could have corrupted the corpus without any visible sign.

1. **A resume bug.** Flags came back from Parquet as a numpy array of a list when we wrote them to a checkpoint. The translation routine considered anything that was not a list or a tuple as a failed row. Even though the text was still there, every row that was resumed got an empty_output stamp. This error plagued 700 out of the 4,788 rows in task 3. The run summary looked okay because it counted empties from the text but a quick check of the flag column showed a different story. Now the empty flag is taken directly from the translation so the empty flag is always correct by design. A regression test that resumes a run now checks that no row comes back flagged. We added a rebuild option that re-derives the splits, from existing checkpoints without loading any model, which repaired the 700 rows without touching the GPU.
2. **A silent CPU fallback.** When a run stalled and was killed, the driver went into a state where the GPU was marked as unavailable in a process. The work then kept translating on the CPU. Float32 arithmetic gives results from float16 arithmetic, so a corpus that is partly generated with Float32 and partly with Float16 cannot be compared within a single decoding fingerprint. Out of 654 sentences that were regenerated on the GPU, two were different from the versions that were produced on the CPU; the other 652 sentences were byte-identical between the GPU and CPU processes. The translator now refuses to auto-select the CPU. The device is recorded in the run manifest.
3. **A stalled batch.** One batch of eight sentences stayed in the beam-search loop for 47 minutes while the GPU was idle and the checkpoint remained untouched. Every batch now must finish before a deadline. If a batch stalls, it is treated like an out of memory error: the batch is halved, retried and isolated. If a single row still overruns, that row is skipped, flagged and reported while the seven neighbouring rows keep their translations.
4. **A report overwrite.** A run restricted to one direction used to overwrite the whole task's report with its own single row. The report now merges by direction, and only within one decoding fingerprint.

4.7 TRAINING IMPLEMENTATION

4.7.1 CONFIGURATION AND REGISTRY

Every training run is described by a YAML file. That file is hashed, and the hash is recorded along with the output. A model registry connects encoder names to checkpoints and tokenizers. Therefore, adding a source language or a new encoder only requires a change in configuration, not in code. This was tested during the source-language comparison, where two new source languages were added without changing any code. A consistency guard checks each configuration against the same-encoder Hindi configuration. This ensures that differences in results are not mistakenly attributed to language differences, but to actual changes in the training recipe. The hyperparameters are listed in Table 4.4.

Table 4.4: Fine-tuning configuration

Setting	Task 2	Task 3
Maximum sequence length	192	64
Batch size × accumulation	16 × 1 (mBERT 8 × 2)	16 × 1
Optimiser	AdamW	AdamW
Learning rate	2 × 10^-5	3 × 10^-5
Weight decay	0.01	0.01
Warm-up	10% of steps	10% of steps
Maximum epochs, early-stopping patience	20, 5	40, 8
Development, test fraction	0.10, 0.10	0.15, 0.15
Precision	fp16 mixed	fp16 mixed
Seeds	3	3

mBERT on task 2 uses micro-batches of 8 with two-step gradient accumulation, forced by GPU memory. This keeps the effective batch at 16 and leaves the gradient unchanged, because the loss is a mean.

4.7.2 MEMORY BUDGETING

A budgeting script measured the batch that can survive a forward pass, a backward pass, and an optimiser step for each encoder. The fixed cost of full-precision weights, gradients and the two AdamW moment buffers equals 16 bytes per parameter of batch size or sequence length. Table 4.5 shows the result. XLM-R base needs 4.14 GiB for

this fixed cost while only 3.68 GiB of usable device memory is available so XLM-R cannot be fine-tuned at any batch size. An adapter-based workaround was tested and rejected, because it would disturb the pre-training-only comparison that the encoder study is designed to provide. XLM-R's configuration will be kept for future runs on larger hardware.

Table 4.5: Memory budget per encoder at maximum length 128 with fp16

Encoder	Maximum batch	Peak (GiB)	Fixed cost (GiB)	Note
IndicBERT	64	3.04	0.50	Fits
mBERT	48	3.32	2.65	Fits
XLM-R base	0	not run	4.14	Does not fit at any batch size

4.7.3 CONVERGENCE CHECK

Three of the nine IndicBERT runs on task 3 never fit their training data. IndicBERT runs test scores look like a runs on a task, at its data ceiling. Convergence is judged on the training fold. A IndicBERT run whose peak training macro-F1 stays below 0.5 is marked as not trained. IndicBERT runs remain in all all-seeds figures. The source comparison excludes IndicBERT runs. A IndicBERT model that never moved from its initialisation scores badly at home and abroad. The IndicBERT model shows a zero transfer gap that ranks it as the best source.

4.7.4 INFERENCE AND LOGGING

The inference path loads a checkpoint in read-only mode and checks that no optimiser is attached and that no gradient step is performed. It logs each prediction with the item identifier, seed, gold label predicted label and class probabilities. Instances that are correct in the source language but wrong in the target are marked by a mismatch module. Then those instances are exported by a failure module as input to the diagnostic phase.

4.8 DIAGNOSTIC IMPLEMENTATION

The router reuses the corpus similarity module for its gate. It creates a label file and a audit trail for each module. The fragmentation module counts tokens using the encoder's tokenizer. The morphology module relies on a ladder for fallback because a

Universal Dependencies treebank for Malayalam does not exist. The first fuzzy-match rule, which used an edit distance of two, gave 96 percent false hits. This happened because of short stems, like the one for water, matching two-letter stems. That rule was replaced with the stem-match rule from Chapter 3. This change cut masking labels from 10,642 to 2,484. Failures per cell dropped to between 2.0 and 7.8 percent. The saliency module computes Integrated Gradients using a zero-embedding baseline and includes a completeness check. Two cells needed 800 integration steps to converge. One cell, IndicBERT on task 3 stayed at 8.2 percent unconverged, which is above the 5 percent limit. That case is marked as open. Divergence thresholds were set for each cell based on the percentile of correctly handled pairs. The values are: 0.038 for task 2 IndicBERT, 0.047 for task 2 mBERT, 0.085 for task 3 IndicBERT, and 0.078 for task 3 mBERT.

The behaviour module implements the four data and model labels of Stage 3. Before adding the behaviour module, 88 to 91 percent of failures had no attribution. After adding the behaviour module, the unattributed failures dropped to 9.2, 19.7 11.8 and 14.4 percent for the four combinations of task and encoder. The aggregation phase counts the labels. It does not change any of them. The report is generated from the artefacts in check mode, like the datasheet, and a re-run produces exactly the same output byte-for-byte.

4.9 USER INTERFACE: RESULTS DASHBOARD

A Streamlit dashboard lets a reader explore the results without needing a GPU. It has three pages. You switch between them using the task option in the sidebar. The transfer page shows one heatmap for each model showing transfer gaps. The "causes" page breaks down what each setup’s failures are made of. It shows this for machine-translated text and you can view it by group or by each individual label. The explorer page shows one failed sentence at a time. For each, you see the source and target text, every seed’s prediction for both models, the cause label and the result of every check that led to that cause. It only reads artefacts that already exist. It does not load any model.

4.10 REPRODUCIBILITY MEASURES

All corpus data is stored in Parquet files. If one configuration is run twice, the output will be the same and every artefact records its configuration hash. Upstream data is

saved with SHA-256 checksums. The release can only be read. It contains a manifest. The translation environment cannot change. Long jobs should be run by a person manually. They are designed to save progress and continue later rather than to be left unattended, because each job takes a long time and relies on GPU access that is often unavailable.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 INTRODUCTION

This chapter talks about what was tested and also what the results were. It starts with the testing plan and then the measurement method used. Then it goes through the goodness of the dataset, performance in the language, how well the model worked without any extra training and how different encoders, source languages and directions compared. It also inspects the reason why some things did not work well. After that it checks the results, compares them to numbers from studies, and discusses the findings and their limits. Unless negated all numbers are averages from three runs, with the spread shown. The score used is macro-F1.

5.2 TESTING STRATEGY

Every source module comes with automated tests. Only after a task's acceptance criteria pass as a check and not by inspection is a task marked done. Table 5.1 lists the main test groups and what each one asserts.

Table 5.1: Automated tests by module

Module	Tests	What is asserted
entities	32	At least five positive and five negative fixtures per check; equivalent quantities across scale words and scripts compare equal; corrupted rows are not reported as numeral loss
spans, generate	33	A recovered span slices the number out of the translation; 22, 2019 reads as two numbers; every source row yields one translated row with the same identifier and label; row loss, reordering and an unflagged empty translation are refused
integrity	15	A deliberately script-leaked fixture is caught; the danda is not leakage; a leak counts only when the source is clean; flags written twice equal flags written once
labse_gate	7	Known-similar and known-dissimilar pairs fall on the expected side of τ, tested on the real model
freeze	7	The manifest verifies; a modified, a deleted and an unexpected extra file each fail verification
conditions	9	No split is both training source and evaluation target; training and evaluation items are disjoint
datasheet	11	Figures are read back from artefacts; stated limitations stay stated; the check mode detects staleness
findings	20	Phase 7 shares reconcile row for row with the failure files

Module	Tests	What is asserted
Phase 6 modules	68	Router, fragmentation, morphology and saliency (fast tests, CPU only)

5.3 EVALUATION METRICS

Macro-F1 is the primary classification criteria. Accuracy and per-class F1 are also reported along with it.

Transfer gap is defined as the source macro-F1 minus the target macro-F1 calculated on the items (Equation 3.2). Most evaluated items are the target versions of sentences that the source model saw during training. Hence, the source in language term acts as a training-text ceiling. It does not serve as a held-out estimate. Therefore, transfer gap measures the cost of changing the language while keeping sentence difficulty the same. Transfer gap is not a measure of generalisation to content that has not been seen.

Confidence intervals are 95 percent intervals from a 1,000-sample bootstrap over items, computed per seed.

Corpus quality measures are the entity preservation rate, the LaBSE similarity and the share of rows below τ, the counts of each integrity finding, and for task 1 the span recovery rate.

Failure share is the fraction of a group's failures that carry a label, with a cluster bootstrap interval.

5.4 CORPUS QUALITY

5.4.1 STRUCTURAL RESULTS

All three corpora pass every blocking check. Each block joins four ways, with 10,640, 6,130 and 6,016 items for task 1, 2,238, 2,228 and 2,072 for task 2, and 532 for each block of task 3. No translated row carries a label different from its source's, and no translation came back empty. Table 5.2 summarises the non-blocking findings.

Table 5.2: Translation quality by task

Measure	Task 1	Task 2	Task 3
Machine-translated rows	68,358	19,614	4,788
Placeholder leakage	3,524 (5.2%)	18 (0.1%)	4 (0.1%)
Escape leakage	734 (1.1%)	1 (<0.1%)	1 (<0.1%)

Measure	Task 1	Task 2	Task 3
Script leakage	421 (0.6%)	3 (<0.1%)	17 (0.4%)
Suspected truncation	1,141 (1.7%)	2 (<0.1%)	0
Below τ (translation_drift)	11,419 (16.7%)	357 (1.8%)	85 (1.8%)
τ	0.82 (inherited)	0.82	0.73
Median LaBSE, range over directions	0.844 to 0.937	0.902 to 0.949	0.859 to 0.908
Entity preservation, range over directions	49.8 to 78.7%	89.5 to 95.7%	77.4 to 97.0%

Task 1 is much worse than the other two tasks in every measure. This makes sense, because Task 1 is a task that has a lot of numbers, named entities, and nukta characters. These are the things that the two corruption modes affect. In Task 1, the common corruption type is unrestored entity placeholders. They show up in 5.2 percent of the rows. Script leakage usually happens when a brand or domain name stays in the script and is not translated. This is a translation mistake, not a problem with encoding.

5.4.2 DO NUMERALS SURVIVE TRANSLATION?

Task 1 measures survival directly, as shown in Table 5.3 and Figure 5.1. The annotated numeral is recovered in 82.8 percent of translations. On every recovered row, the recovered offsets slice out the annotated numeral precisely. When we re-checked the rows using value-level entity comparison, we were able see the types of loss separately. In 9.7 percent of rows, the output keeps digits, but not the annotated numeral. In 3.7 percent of rows, the output is corrupted. In 3.3 percent of rows the, output contains no digits all. 0.4 percent of rows have the same quantity, just rescaled. In 30 rows or 0.4 percent, the quantity survived, but the recovery missed it, which limits the error of the recovery procedure itself.

Table 5.3: Survival of the annotated numeral in task 1 machine-translated rows

Outcome	Rows	Share
Span recovered	56,624	82.8%
Digits present, annotated value gone	6,618	9.7%
Output corrupted (placeholder or escape)	2,560	3.7%
No digits in the output	2,234	3.3%
Same quantity, rescaled	292	0.4%
Quantity preserved, span missed	30	0.04%

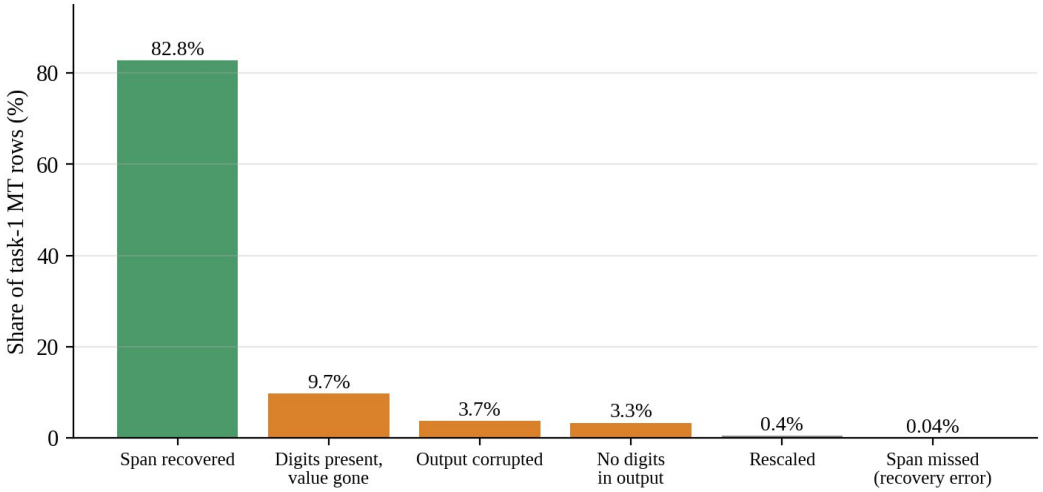


Figure 5.1: Outcome of the annotated numeral in task 1 machine translation

About one out of every six financial numerals does not remain unchanged during Indic-to-Indic translation. A stricter count includes every currency symbol and percentage that appears in a sentence, not just the one marked. Rows that are corrupted, which represent 6.2 percent of task 1, are omitted from this calculation. Still, no direction in task 1 reaches the 95 percent threshold: the rates vary from 49.8 percent for the Bengali-to-Malayalam direction, to 78.7 percent for the Telugu-to-Hindi direction (see Figure 5.2). Directions in task 2 range from 89.5 percent to 95.7 percent, while directions in task 3 range from 77.4 percent to 97.0-percent. In each set, the weakest direction is Hindi-to-Malayalam.

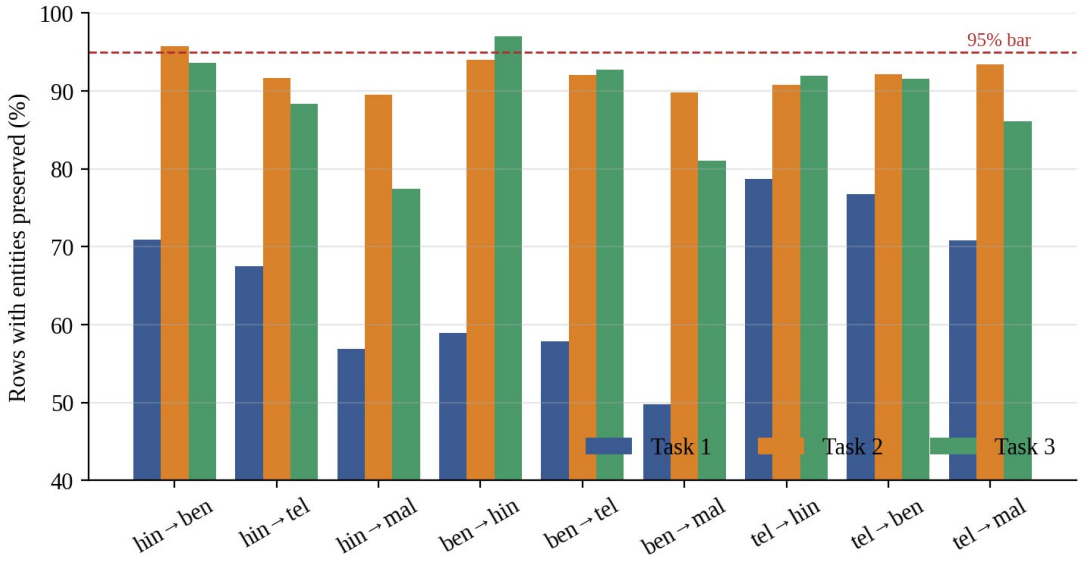


Figure 5.2: Entity preservation by direction and task, with the 95 percent bar

5.4.3 SEMANTIC SIMILARITY BY DIRECTION

In the LaBSE similarity analysis, task 1 is split by source language (see Table 5.4, Figure 5.3). Hindi-source directions show a similarity ranging from 0.906 to 0.937, and they dip below τ on between 5 and 11 percent of the rows. Bengali-source directions show a similarity ranging from 0.844 to 0.854, and they dip below τ on between 29 and 34 percent of the rows. This pattern shows up in task 1, the only task that has blocks with different sentences so it reflects the source texts as much as the translation directions.

Table 5.4: Median LaBSE similarity and share below τ by direction

Direction	T1 median	T1 below τ	T2 median	T2 below τ	T3 median	T3 below τ
hin→ben	0.935	6.5%	0.950	0.3%	0.908	0.4%
hin→tel	0.937	4.7%	0.945	0.2%	0.903	0.8%
hin→mal	0.906	10.9%	0.908	4.4%	0.859	4.5%
ben→hin	0.854	29.4%	0.931	0.8%	0.898	0.4%
ben→tel	0.849	31.5%	0.923	2.4%	0.889	1.1%
ben→mal	0.844	34.1%	0.902	5.4%	0.870	2.1%
tel→hin	0.916	15.5%	0.943	0.4%	0.891	1.1%
tel→ben	0.909	19.8%	0.939	0.7%	0.881	1.9%
tel→mal	0.906	18.8%	0.924	1.6%	0.861	3.8%

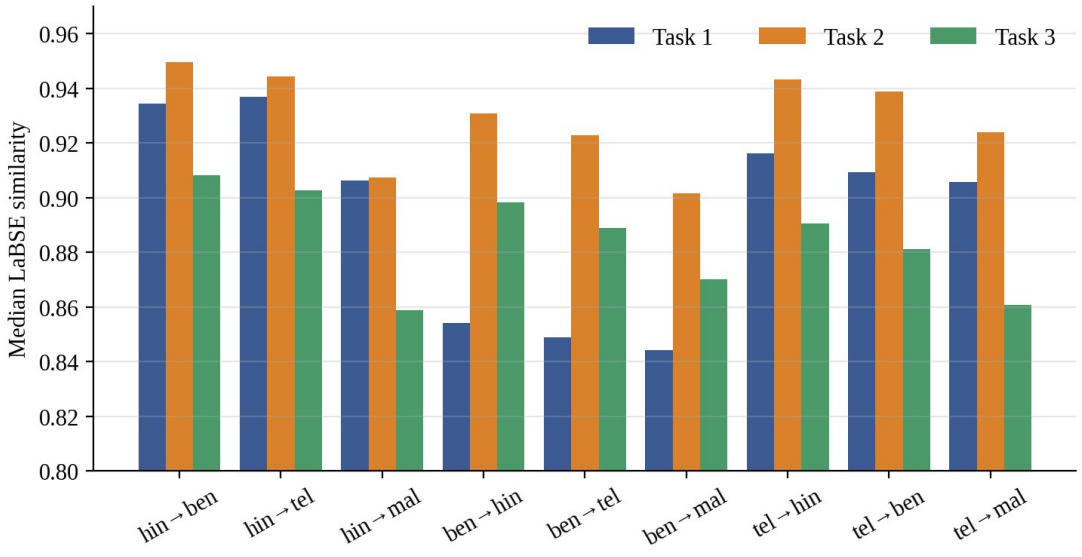


Figure 5.3: Median LaBSE similarity by translation direction and task

5.4.4 CALIBRATING THE THRESHOLD

Human translations in task 2 and task 3 allow us to check the threshold. In task 2, the median score for pairs is 0.921 and the 5th percentile is 0.846. With τ equal to 0.82, every human translation in task 2 passes the threshold. In task 3, the median score is 0.870 and the 5th percentile is 0.729. Using τ equal to 0.82 would cause 23.4 percent of translations in task 3 to fail the threshold. The reason for the difference is the length of the text. Task 2 has a length of 148 characters while task 3 has a median length of 86 characters. LaBSE cosine similarity decreases when the context becomes shorter. Therefore, τ is set separately for each task as shown in Table 5.5. When τ is calibrated to 0.73, the number of drift-flagged rows in task 3 drops from 740 to 85.

Table 5.5: Calibrated similarity threshold by task

Task	τ	Basis
Task 1	0.82	Inherited, not calibrated, because no human reference exists in any direction
Task 2	0.82	Confirmed: rejects 0.0% of human pairs and no direction exceeds the 20% trigger
Task 3	0.73	Revised: the value that accepts 95% of human pairs

A second result qualifies every similarity figure in this report. Machine translation scores higher than translation of the same items in both calibratable tasks and in every language pair. On task 2 for example Hindi to Bengali machine translation scores 0.950 against 0.918 for the human Bengali and Hindi pair;, on task 3 the figures are 0.908 and 0.876. LaBSE cosine rewards translation so it ranks directions usefully but is a poor absolute measure of adequacy.

5.4.5 RANKING THE DIRECTIONS

Ranking the nine directions based on drift, entity preservation, corruption and truncation using the mean of each measure’s rank gives one result across all tasks. Translating into a Dravidian language shows more drift than translating from Telugu back into an Indo-Aryan language. This difference is 2.6 points on task 1 (20.3 percent versus 17.6 percent), 2.6 on task 2 (3.1 percent versus 0.5 percent), and 0.6 on task 3 (2.1 percent versus 1.5 percent at its calibrated τ). Two other comparisons switch in direction between tasks, whether Hindi to Malayalam drifts more than Bengali to

Malayalam, and whether Telugu-source directions drift more than Indo-Aryan-source ones. Neither of these is reported as a feature of the language pair.

5.5 IN-LANGUAGE BASELINES

Table 5.6 shows the in-language macro-F1 on each held-out test fold. Figure 5.4 shows the task 2 comparison with the published figures. Five of the six IndicBERT baselines are at or above IndicFinNLPs published figure. The exception, task 2 Hindi, is 3.7 points below. The training behaviour is healthy; training macro-F1 of least 0.987 on every seed and a test standard deviation of 0.004. The shortfall is because of a test fold that is resampled per seed and to untuned hyperparameters, and not because of a modelling error. Tuning increased the mean by nothing, but made the run-to-run spread ten times tighter which shows the shortfall as a stable property of this reproduction and not noise. The task 3 comparison is not very important; the published 0.05 is the score of a model that predicts one class for every item. Scoring higher than that only shows that the models are better than that.

Table 5.6: In-language macro-F1 on held-out test folds

Task	Language	Published	IndicBERT	mBERT
2	Hindi	0.86	0.823 ± 0.004	0.874 ± 0.017
2	Bengali	0.80	0.820 ± 0.028	0.858 ± 0.014
2	Telugu	0.79	0.819 ± 0.015	0.860 ± 0.009
3	Hindi	0.05	0.153 ± 0.025	0.335 ± 0.042
3	Bengali	0.05	0.127 ± 0.055	0.292 ± 0.074
3	Telugu	0.05	0.095 ± 0.047	0.243 ± 0.036

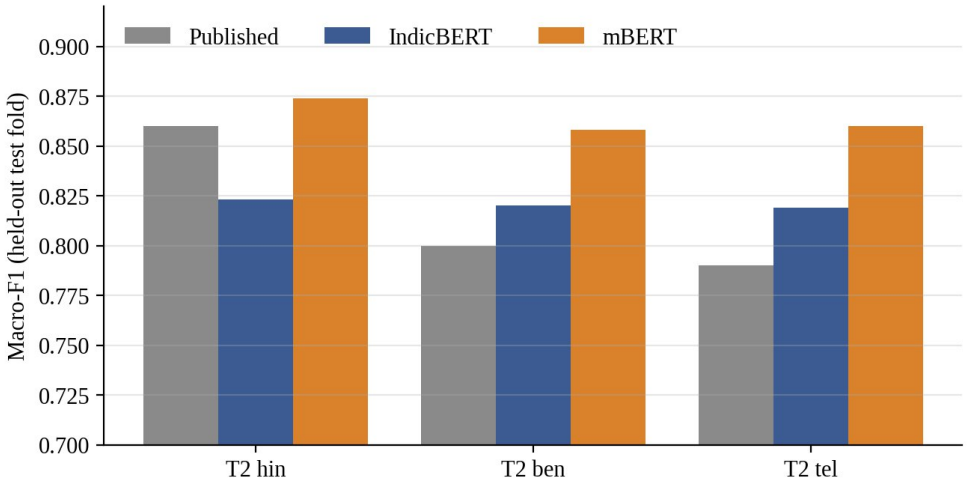


Figure 5.4: In-language macro-F1 for task 2 against the published figures

Three of the nine task 3 IndicBERT runs did not fit their training data: IndicBERT with Bengali seed 0 and IndicBERT with Telugu seeds 0 and 2 reached a peak below 0.5 for training macro-F1. On a task that has reached its data ceiling, the test scores of those runs look like those of a healthy run. This is why convergence is checked on the training fold. mBERT had no failures under the same settings. These runs still appear in every all-seeds figure. Over converged seeds, task 3 Bengali and task 3 Telugu score 0.159 and 0.150.

5.6 ZERO-SHOT TRANSFER

Two properties of the measurement matter when reading the transfer results. First, the in-language score that the gap subtracts from is computed on the evaluation items in the source language. 80 Percent of the items on task 2 and 70 percent on task 3 are in the source model's training fold. This represents a training-text ceiling. For task 2 in Hindi with IndicBERT, the score reaches 0.954, compared to 0.823 on the held-out fold. Second, task 3 is at its data ceiling with 532 headlines spread across ten classes.

Figures 5.5 and 5.6 show the gap for every source and every target. Appendix 4 gives the table with seed spread. Transfer costs a large share of in-language performance in every cell. On task 2 the gaps range from 0.26 to 0.45 with IndicBERT and from 0.11 to 0.24 with mBERT. Out of Hindi, IndicBERT loses 0.39 to 0.45 on task 2 and 0.50 to 0.51 on task 3. On task 3 the gaps stay between 0.46 and 0.56 no matter the source, target, or encoder.

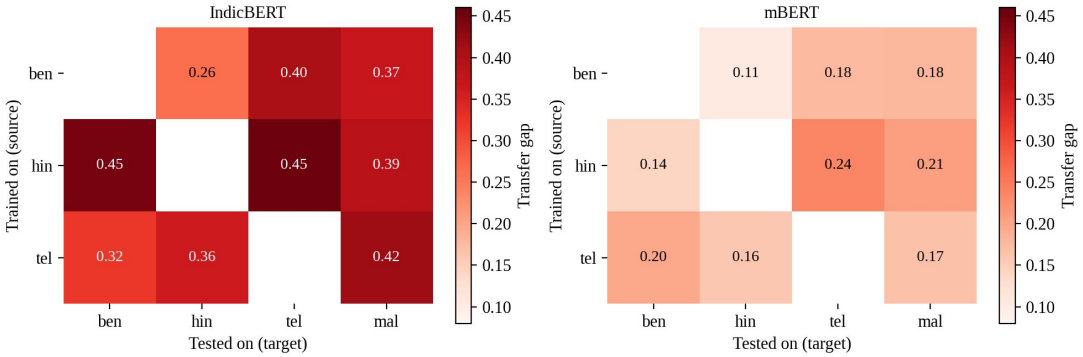


Figure 5.5: Task 2 transfer gap, source on the vertical axis and target on the horizontal axis

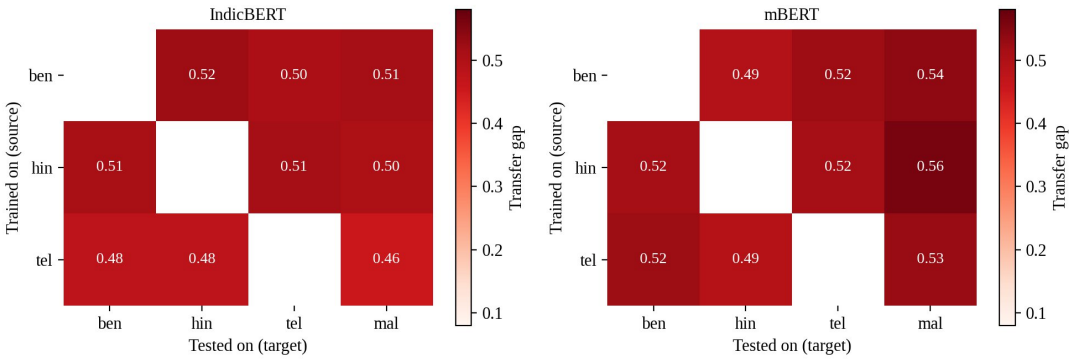


Figure 5.6: Task 3 transfer gap, source on the vertical axis and target on the horizontal axis

5.6.1 IS THE LOSS AN ARTEFACT OF THE TRANSLATIONS?

When we evaluate using machine-translated target text from the sources block instead of native text, the gaps shift by only a few points at most. For the Hindi source and IndicBERT on task two the gaps are between 0.38 and 0.47 on machine-translated targets and between 0.39 and 0.45 on targets. Thus the loss is not an artefact of this project's translations. If translation damage had been the driver, evaluation on genuine human-written Bengali would have scored noticeably better than on machine-translated Bengali but it did not.

5.6.2 TRANSLATIONESE CONDITIONS

The translationese conditions keep the language the same and only change where the text comes from. When using IndicBERT on task 2, a Hindi, Bengali or Telugu model scores between 5.7 and 7.8 points lower on machine-translated text in its language compared to native text. On task 3 the drop is even bigger ranging from 8.8 to 26.6 points. The native arm refers to the model's split, which is mostly made up of its training data. The translated arm, however, is not. Because of this the differences seen here are a bound on the cost of using translated input. They are not a measure of it.

5.7 ENCODER COMPARISON

mBERT beats IndicBERT on the target language in every one of the twelve Hindi-source cells, six cells for each task on both translated targets as shown in Table 5.7. The margin is between 0.15 and 0.37 macro-F1. This margin is larger than the combined seed spread of the two encoders in every cell. mBERT also achieves higher scores in-language on held-out data. The two tasks differ in how the performance changes. For task two, mBERT starts at a higher level and loses less when transferring.

The gaps run from 0.12 to 0.24 for mBERT against 0.38 to 0.47 for IndicBERT. For task three, both encoders lose the same amount. The gaps are 0.45 to 0.56 for mBERT versus 0.50 to 0.52 for IndicBERT. Therefore, the report does not say that the better encoder always transfers better.

Table 5.7: Macro-F1 by encoder, Hindi source

Measure	IndicBERT	mBERT
Task 2, in-language (held-out)	0.823 ± 0.004	0.874 ± 0.017
Task 2, target (mean of 6 cells)	0.519	0.787
Task 2, transfer gap (range)	0.38 to 0.47	0.12 to 0.24
Task 3, in-language (held-out)	0.153 ± 0.025	0.335 ± 0.042
Task 3, target (mean of 6 cells)	0.087	0.294
Task 3, transfer gap (range)	0.50 to 0.52	0.45 to 0.56

Tokenisation, which strongly affects per-language performance [29], does not explain the result. IndicBERT needs 14 to 24 percent fewer tokens than mBERT for the same sentences and IndicBERTs advantage is greatest for Malayalam, yet IndicBERT underperforms on every measure. Capacity might explain IndicBERTs performance. IndicBERT shares one layer's weights across all twelve layers so IndicBERTs transformer body has seven point eight million parameters versus mBERTs point six million. Separating capacity from breadth requires a comparison of mBERT against XLM-R, which share an identically sized transformer body and differ mainly in vocabulary and pretraining breadth. That comparison could not be run on the hardware so the cause remains open.

5.8 SOURCE LANGUAGE AND DIRECTION

Training on Hindi gives the largest mean gap on task 2 with both encoders, and Bengali the smallest (Table 5.8, Figure 5.7). The three source models are equally good in their own language, within a quarter of a point of each other, so the ranking concerns transfer and not a stronger model.

Table 5.8: Mean transfer gap by source language and target family, task 2

Group	IndicBERT	mBERT
Source: Bengali	0.345	0.153
Source: Telugu	0.367	0.174
Source: Hindi	0.429	0.196

Group	IndicBERT	mBERT
Target family: Indo-Aryan	0.349	0.151
Target family: Dravidian	0.406	0.194

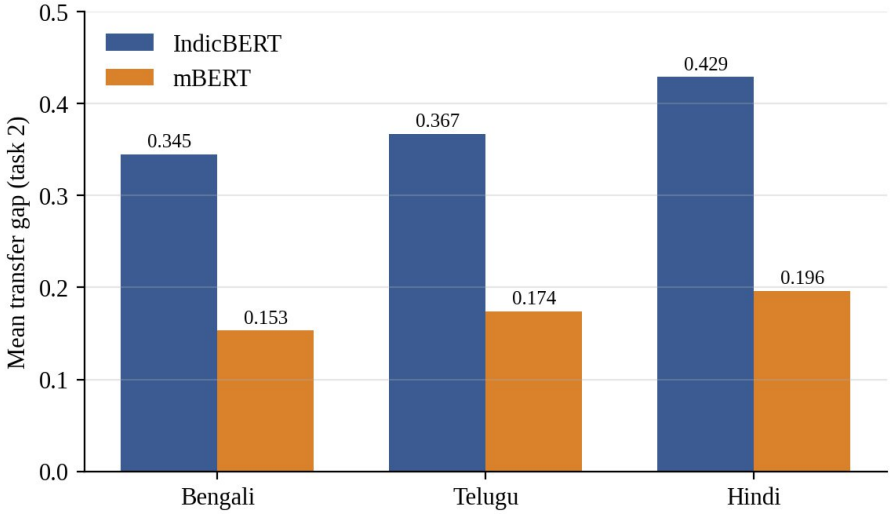


Figure 5.7: Mean transfer gap by source language on task 2

The cost is mostly in the target language. When translating into a language the mean gap is 0.406 with IndicBERT and 0.194 with mBERT. This compares to 0.349 and 0.151 when translating into a Aryan language. The corpus quality results in Section 5.4 showed the trend in translation quality measured using a different method. This agreement is the result in the study. Transfer is also not equal in both directions. Hindi to Bengali loses more than Bengali to Hindi on both encoders beyond what might be explained by seed noise: 0.448 versus 0.263 with IndicBERT and 0.142 versus 0.105, with mBERT.

Typological closeness does not consistently find the source. A source from the family is best for one out of three targets with IndicBERT and for all three targets with mBERT. The two encoders also have opinions about the best source for Malayalam: Bengali with IndicBERT and Telugu with mBERT. Task 3 does not support any source comparison because there is no difference, between sources and no direction asymmetry that goes beyond seed noise.

5.9 FAILURE ATTRIBUTION

The router labelled every failure in all 72 cells covering two tasks, two encoders and 18 transfer conditions. Table 5.9 and Figure 5.8 display the details of those failures.

Between nine point two and nineteen point seven percent of the failures are unattributed. Linguistic labels make up eight point eight to twelve point zero percent while data or model labels cover sixty-nine point zero to seventy-nine point four percent. Most failures are therefore described by their location than by a feature of the target sentence. They appear in every target language in one text provenance on only some seeds or, under a collapsed classifier. These data and model labels reveal a pattern. They do not name a mechanism.

Table 5.9: Composition of transfer failures, share of failures

Cell	Failures	Unattributed	Linguistic	Data	Model
Task 2, IndicBERT	19,323	9.2%	12.0%	44.6%	34.2%
Task 2, mBERT	9,714	19.7%	11.3%	48.3%	20.8%
Task 3, IndicBERT	6,102	11.8%	8.8%	72.2%	7.1%
Task 3, mBERT	7,103	14.4%	9.4%	64.9%	11.2%

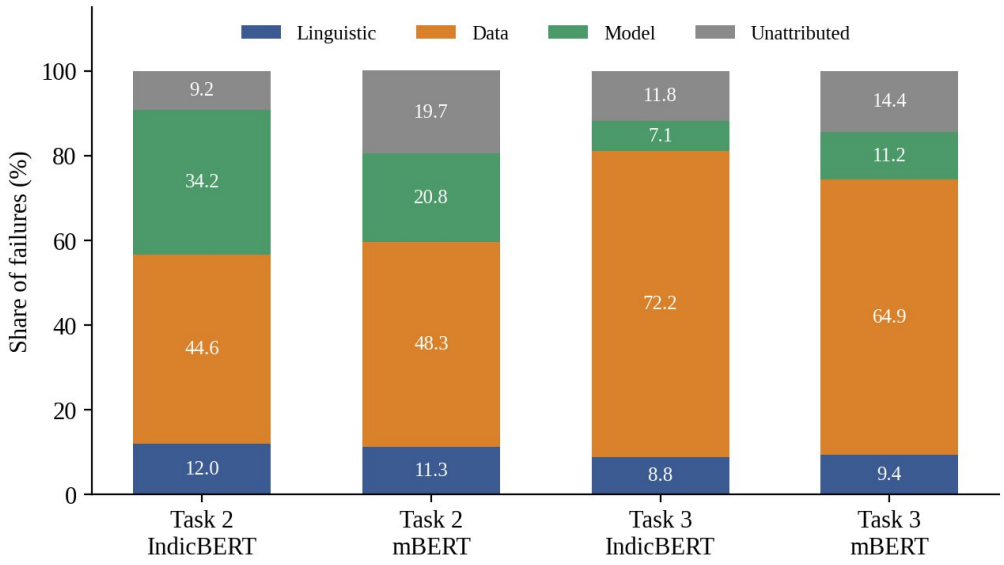


Figure 5.8: Composition of transfer failures by task and encoder

Two of the modules failed to validate. Items that contain a masked term do not fail more often than items that keep the term intact: masked items fail more often in only 20 out of 44 cells. Saliency divergence does not separate failures from handled pairs; the area under the curve is between 0.48 and 0.52. Tokenizer fragmentation never fires. At most, 0.5 percent of failures involve tokenizer fragmentation because a ratio of 2.0 is rarely reached. Labels from these modules are reported. They should not be read as established causes, however.

Of 228 group comparisons, 22 meet the finding rule, and 11 of those could be produced by a known measurement limit on its own. The remaining 11 fall into three groups.

Direction (task 2). Failures out of Telugu carry more model-behaviour labels than failures into it, on both encoders (40.2 percent against 34.4 percent with IndicBERT, 24.9 percent against 14.4 percent with mBERT), mainly through seed instability.

Source. On task 2, when the target languages are Bengali and Malayalam, failures coming from Telugu sources have more model-behaviour labels compared to those from Hindi sources. The numbers show 46.8 percent for Telugu-source failures versus 18.4 percent for Hindi-source ones. Similarly, for the comparison, it's 22.0 percent versus 14.7 percent. On task 3, Hindi-source failures have more terminology_gap labels than Telugu-source failures. This label comes from a module that did not pass validation. So the result is not reliable.

Encoder. 27.3 percent of IndicBERT’s failures on task 2 are due to class collapse. This issue never happens with mBERT. IndicBERT also shows more cross_lingual_hard failures on both tasks. The numbers are 28.5 percent versus 24.4 percent on task 2 and 60.6 percent versus 46.6 percent on task 3. Meanwhile, mBERT’s failures are often seed_unstable with 19.0 percent compared to 6.3 percent on task 2 and 10.5 percent versus 5.0 percent on task 3. MBERT also has more provenance_specific failures, at 25.5 Percent versus 15.9 percent on task 2 and 16.6 percent versus 8.7 percent on task 3.

Hindi-source failures happen often than Bengali-source failures. In task 2, 32.9 percent of the evaluated rows fail with IndicBERT compared to 27.7 percent for Bengali. With mBERT the numbers are 17.0 percent for Hindi and 13.4 percent for Bengali. Still, there is no difference in what causes these failures. Hindi fails more, but not in a different way. When looking at machine-translated targets, between 9.5 and 25.0 percent of failures are marked as translation_drift or provenance_specific. This is the closest we get to saying that the project’s own translation led to the failure. The only differences between translated inputs that pass the rule are in translation_drift. This is just an effect of the measurement method. The τ measure is designed to accept 95 percent of translations. Also, LaBSE gives scores to literal translations than to human translations, which skews the results. If we repeat every comparison but leave out the

task 3 runs that never trained, 20 verdicts change. Only one of those changes flips between a finding and not a finding.

5.10 VALIDATION

Several independent checks support the reliability of the results.

Baseline gate. Each source model was compared with the published, in-language figure before any transfer number was read. Five of the six IndicBERT baselines exceeded the published figure. The sixth, task 2 Hindi, fell short. The gate passed only with a written diagnosis.

Agreement between unrelated measures. Splitting task 1 directions by whether the annotated numeral survived separates two groups. For example, in Telugu to Bengali 8.3 percent of rows that have a recovered span fall below τ while 68.9 percent of rows that lost their span fall below τ. A digit locator and a sentence embedder, which never see each other's output, both agree on which rows were wrong. The into-Dravidian cost is visible in translation drift and in transfer gaps both.

Hand check of the translations. The project maintainer signed off on 20 sentences per translation direction on 10 September 2026, which equals 180 rows per task. This covers 0.3 percent of task 1's machine-translated rows, 0.9 percent of task 2, and 3.8 percent of task 3. Every numeric difference in the sampled translations was also classified manually. This is what exposed the corruption modes.

Hand check of the diagnostic labels. A spot-check of 200 task 2 IndicBERT failures with 50 for each target language matched the router’s label in 197 cases (98.5 percent). Out of the 200, 160 had data or model labels. For the 40 that were linguistic or unattributed, agreement was 37 out of 40. No disagreement stood out as systematic. No threshold was adjusted. The other three combinations of task and encoder were not checked.

Reproducibility. Two runs using the setup produced byte-identical output. Running Phase 7 again gives the findings report. The 72 cells match row for row with the failure files. A tamper test showed that if any file is changed, deleted or added, the frozen release fails verification.

5.11 COMPARISON WITH PUBLISHED AND PRIOR RESULTS

Table 5.6 compared the in-language baselines with IndicFinNLP's published figures. The project's IndicBERT baselines on task 2 are 0.823, 0.820 and 0.819 versus 0.86, 0.80 and 0.79 published. On task 3, the published figure is the score of a one-class predictor, which both encoders exceed.

The transfer results can be compared with the research in the direction but not in the amount. The presence of a transfer gap supports what XTREME [13], [28] found and the bigger loss into more different languages supports what Lauscher et al. [19] said. The result that Hindi turns out to be the worst source for task 2 even after being the language with the resources among Indic languages matches the idea that picking a source language depends on more than just how much data is available [22]. Lastly the warning from Agrawal et al. [2] that machine-translated benchmarks can reflect translation quality was tested here directly. Native targets and machine-translated targets show gaps that are only a few points apart. So for these tasks the gap is mostly not due to translation errors.

5.12 DISCUSSION

Four points follow from the results.

First the corpus audit changed the project. Looking at tasks 2 and 3 as three splits would have meant training and testing, on the sentences without any clear notice. That issue caused the audit, the alignment process and the item-level partition. The same idea also made the human-translation ceiling. This helped make τ a reasonable value.

Secondly translation damage to numerals is very serious, and occurs mostly in tasks where the text is full of numbers. One in six annotated numerals did not survive the translation. In Task 1, no direction meets the 95 percent preservation threshold. A research pipeline that used machine translation without checking for this kind of damage would have thought the corpus was clean. Since the issue is silent and hard to spot the project’s rule of flagging and keeping every row is important. The flagged rows are how we measure the problem.

Third, transfer loss is large. It is mostly not due to translation errors. The numbers look bigger in one way and smaller in another, and are inflated because the in-language ceiling uses training text. They are deflated because the items being evaluated are the

content in both languages so differences in difficulty are removed. The gaps show the cost of language changing.

Fourth, the diagnostic framework did not explain the gap. The causes it listed were mostly descriptions of where errors occurred. The two linguistic modules which were tested against a model did not perform better. This is a result and it is reported as such: we know the size and shape of the gap but we do not know the mechanism, behind it.

5.13 LIMITATIONS AND THREATS TO VALIDITY

No native Malayalam. Every Malayalam result mixes transfer difficulty with translation quality. No human reference exists to calibrate τ for it.

Limited human verification. Human verification looks at a sample of 180 rows per task done by the project maintainer. It is not an audit by native speakers for each target language. The automated checks make sure that numerals, labels, structure and script are preserved. They do not check if every sentence actually means the thing.

Similarity is not adequacy. LaBSE cosine ranking works for directions but it favors literal wording. That means τ and the drift rates are relative, not absolute. For Task 1, τ inherits from work and cannot be recalibrated.

Shared content by design. Training and evaluation share content across languages. Therefore, the gaps are not measures of generalisation to sentences that have not been seen.

Two encoders, not three. XLM-R could not be fine-tuned on a 4 GB GPU, so the controlled test that separates capacity from breadth across languages was not run.

Unconverged runs. 3 out of 9 IndicBERT runs for Task 3 were never trained. We included these in all-seed figures. They are not used in the source comparison. The comparison for Task 3 is weak due to the lack of data.

Diagnostic limits. The shares displayed refer to failures. Two linguistic modules fail to predict failures. The data and model labels only show where failures happen and not why failures happen. One Integrated Gradients cell remains above its convergence limit. The hand check covers one of the four

combinations of tasks and encoders. Some differences are limited by measurement limits, like Hindi's inability to accept the label.

Single seed set and hardware. 3 seeds per configuration give a rough seed spread. A few cells have only one or two converged seeds. Hence, no standard deviation is available for them.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The project asked whether multilingual language models keep meaning without loss when knowledge transfers between Indic languages. For the purpose of answering this question the project built a four-way financial corpus in Hindi, Bengali, Telugu and Malayalam using IndicFinNLP. The corpus contains 92,760 machine-translated rows, each row kept, flagged and none filtered. Every single row is checked for preservation, structural integrity and semantic similarity against a threshold that is calibrated on human translations wherever such translations exist. The independence audit that came before translation changed the design: the sustainability and ESG-theme splits turned out to be the items, in every language while the numeral split is sourced independently.

Financial numerals are fragile under translation. Roughly one annotated numeral in six does not survive translation intact and no direction of the numeral task reaches a ninety-five percent preservation rate for the numerals, currency symbols and percentages in a sentence. Cross-lingual transfer is costly well. With content held fixed IndicBERT loses between zero point two six and zero point four five macro-F1 on sustainability classification and mBERT loses between zero point one one and zero point two four. Evaluating on the project's translations instead of native text changes the gap by only a few points so the loss is not an artefact of the corpus. MBERT outperforms IndicBERT in all twelve Hindi-source cells. The evidence does not support attributing that to tokenisation. On sustainability classification, Hindi, the default choice of source is the source language on both encoders. There the cost sits mainly in the target language. Moving into a Dravidian language costs more than moving into an Indo-Aryan one. The same direction appears in translation quality on all three tasks, measured by a method. The ESG-theme task, with five hundred thirty-two headlines, over ten classes is too small to support any comparison of sources.

The diagnostic framework went through all 72 combinations of task, encoder and condition. It labeled every failure. The labels did not explain why the failures happened. Linguistic labels accounted for between 8.8 and 12.0 percent of the failures. The rest were mostly described by where the failure occurred—like in every target language on

some seeds or under a collapsed classifier. Two linguistic modules— masking and terminology gap—did not predict failure better than random chance. As a result the report confidently describes the size and shape of the transfer gap. It leaves the causes of the gap open, for further study.

According to the objectives in Section 1.5 the corpus was built, gated and frozen using a generated datasheet (objectives 1 to 3 and 7). The complete planned transfer grid has 126 results for each task. Was finished (objectives 4 and 5). The router main result is that the reasons for the gap are still mostly unclear (objective 6). Two limits surround all of this. Malayalam results mix transfer difficulty with translation quality. Human verification checks 180 translated rows for each task, not the whole corpus. A single 4 GB GPU also prevented using XLM-R the comparison that would show the difference, between model size and multilingual range.

6.2 FUTURE WORK

Encoders. Running XLM-R with mBERT on better GPU equipment would make it clear if the difference in performance is because of how many languages the model knows, while keeping the size the same. It would also show if mBERT is better because it has power. IndicBERT v2 [9] would do the test for a model that is specific, to Indic languages and uses a full-size transformer structure.

Human verification. A native speaker review of each target language beyond what the current maintainer has provided would show whether the translations are actually correct not just that numbers, labels and script made it through. Automated checks can only confirm that the characters and numbers appear correctly. They cannot tell if the meaning is right. A small test set in Malayalam reviewed by a speaker would make a real difference. It would help to separate Malayalam translations from something that just looks like Malayalam and does not mean the right thing in reality.

Diagnostics. Comparing the failure rate of each label with the prediction rate can show whether the property increases the chance of failure. Shares of failures cannot show the property. The morphology module and the terminology module both need redesign. An intervention test could help, like restoring a term and then measuring how the prediction changes. The numeral module which was left out could be easily built again using the span annotations obtained from Task 1.

Evaluation design. Task 1 has possession of independently sourced data. If we treated detection as a transfer task, it would give a measurement which avoids issues around shared content. Training on one part of the data and testing on the other would provide a held-out estimate adding to the fixed-content gap. Few-shot training, in the target language [19] and translate-train methods [1], [3] could show how much of the gap is closed by a little bit of target language data or translated training data. For the ESG-theme task using paraphrase augmentation—like what was done in IndicFinNLP—might help push performance beyond the data ceiling.

Coverage. IndicTrans2 covers all 22 scheduled languages [11]. Because of that the pipeline can be expanded to Dravidian and Indo-Aryan languages simply by changing its configuration. This will test whether the into-Dravidian penalty remains true for languages than Telugu and Malayalam.

Numeral robustness. Numerals are the fragile content. A aware translation step could help, for example by keeping numerals as placeholders that are checked after decoding. This is an improvement to the corpus pipeline. The failure of the placeholder mechanism, in IndicTrans2, which was noted in Chapter 4 shows that the protection itself must be verified.

REFERENCES

[1] Aggarwal, D., Gupta, V., & Kunchukuttan, A. (2022). IndicXNLI: Evaluating multilingual inference for Indian languages. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing* (pp. 10994–11006). https://doi.org/10.18653/v1/2022.emnlp-main.755

[2] Agrawal, A., Fazili, B., & Jyothi, P. (2024). Translation errors significantly impact low-resource languages in cross-lingual learning. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics* (Volume 2: Short Papers) (pp. 319–329). https://doi.org/10.18653/v1/2024.eacl-short.28

[3] Artetxe, M., Goswami, V., Bhosale, S., Fan, A., & Zettlemoyer, L. (2023). Revisiting machine translation for cross-lingual classification. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6489–6499). https://doi.org/10.18653/v1/2023.emnlp-main.399

[4] Artetxe, M., Ruder, S., & Yogatama, D. (2020). On the cross-lingual transferability of monolingual representations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4623–4637). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.421

[5] Chen, C.-C., Huang, H.-H., Takamura, H., & Chen, H.-H. (2019). Numeracy-600K: Learning numeracy for detecting exaggerated information in market comments. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 6307–6313). https://doi.org/10.18653/v1/P19-1635

[6] Chen, C.-C., Tseng, Y.-M., Kang, J., Lhuissier, A., Seki, Y., Day, M.-Y., Tu, T.-T., & Chen, H.-H. (2023). Multi-lingual ESG impact type identification. In *Proceedings of the Sixth Workshop on Financial Technology and Natural Language Processing* (pp. 46–50). https://doi.org/10.18653/v1/2023.finnlp-2.6

[7] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of*

the Association for Computational Linguistics (pp. 8440–8451). https://doi.org/10.18653/v1/2020.acl-main.747

[8] Conneau, A., Rinott, R., Lample, G., Williams, A., Bowman, S. R., Schwenk, H., & Stoyanov, V. (2018). XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 2475–2485). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1269

[9] Doddapaneni, S., Aralikatte, R., Ramesh, G., Goyal, S., Khapra, M. M., Kunchukuttan, A., & Kumar, P. (2023). Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (Volume 1: Long Papers) (pp. 12402–12426). https://doi.org/10.18653/v1/2023.acl-long.693

[10] Feng, F., Yang, Y., Cer, D., Arivazhagan, N., & Wang, W. (2022). Language-agnostic BERT sentence embedding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics* (Volume 1: Long Papers) (pp. 878–891). https://doi.org/10.18653/v1/2022.acl-long.62

[11] Gala, J., Chitale, P. A., AK, R., Gumma, V., Doddapaneni, S., Kumar, A., Nawale, J., Sujatha, A., Puduppully, R., Raghavan, V., Kumar, P., Khapra, M. M., Dabre, R., & Kunchukuttan, A. (2023). IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages. *Transactions on Machine Learning Research.* https://arxiv.org/abs/2305.16307

[12] Ghosh, S., Maji, A., Narayana, A., & Naskar, S. K. (2024). IndicFinNLP: Financial natural language processing for Indian languages. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation* (LREC-COLING 2024) (pp. 9010–9018). ELRA and ICCL. https://aclanthology.org/2024.lrec-main.789/

[13] Hu, J., Ruder, S., Siddhant, A., Neubig, G., Firat, O., & Johnson, M. (2020). XTREME: A massively multilingual multi-task benchmark for evaluating cross-lingual generalisation. In *Proceedings of the 37th International Conference on Machine Learning* (PMLR Vol. 119, pp. 4411–4421). https://proceedings.mlr.press/v119/hu20b.html

[14] Huang, A. H., Wang, H., & Yang, Y. (2023). FinBERT: A large language model for extracting information from financial text. *Contemporary Accounting Research*, 40(2), 806–841. https://doi.org/10.1111/1911-3846.12832

[15] Jørgensen, R., Brandt, O., Hartmann, M., Dai, X., Igel, C., & Elliott, D. (2023). MultiFin: A dataset for multilingual financial NLP. In *Findings of the Association for Computational Linguistics:* EACL 2023 (pp. 894–909). https://doi.org/10.18653/v1/2023.findings-eacl.66

[16] Joshi, P., Santy, S., Budhiraja, A., Bali, K., & Choudhury, M. (2020). The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 6282–6293). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.560

[17] Kakwani, D., Kunchukuttan, A., Golla, S., N.C., G., Bhattacharyya, A., Khapra, M. M., & Kumar, P. (2020). IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020* (pp. 4948–4961). https://doi.org/10.18653/v1/2020.findings-emnlp.445

[18] Kunchukuttan, A. (2020). *The IndicNLP library* [Computer software documentation]. https://github.com/anoopkunchukuttan/indic_nlp_library/blob/master/docs/indicnlp.pdf

[19] Lauscher, A., Ravishankar, V., Vulić, I., & Glavaš, G. (2020). From zero to hero: On the limitations of zero-shot language transfer with multilingual transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 4483–4499). https://doi.org/10.18653/v1/2020.emnlp-main.363

[20] Lee, J., Stevens, N., & Han, S. C. (2025). Large language models in finance (FinLLMs). *Neural Computing and Applications*, 37(30), 24853–24867. https://doi.org/10.1007/s00521-024-10495-6

[21] Libovický, J., Rosa, R., & Fraser, A. (2020). On the language neutrality of pre-trained multilingual representations. In *Findings of the Association for Computational Linguistics: EMNLP 2020* (pp. 1663–1674). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.findings-emnlp.150

[22] Lin, Y.-H., Chen, C.-Y., Lee, J., Li, Z., Zhang, Y., Xia, M., Rijhwani, S., He, J., Zhang, Z., Ma, X., Anastasopoulos, A., Littell, P., & Neubig, G. (2019). Choosing transfer languages for cross-lingual learning. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3125–3135). https://doi.org/10.18653/v1/P19-1301

[23] Mabokela, K. R., Celik, T., & Raborife, M. (2023). Multilingual sentiment analysis for under-resourced languages: A systematic review of the landscape. *IEEE Access*, 11, 15996–16020. https://doi.org/10.1109/ACCESS.2022.3224136

[24] Nag, A., Samanta, B., Mukherjee, A., Ganguly, N., & Chakrabarti, S. (2023). Transfer learning for low-resource multilingual relation classification. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 22(2), 1–24. https://doi.org/10.1145/3554734

[25] Pfeiffer, J., Vulić, I., Gurevych, I., & Ruder, S. (2020). MAD-X: An adapter-based framework for multi-task cross-lingual transfer. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 7654–7673). https://doi.org/10.18653/v1/2020.emnlp-main.617

[26] Pires, T., Schlinger, E., & Garrette, D. (2019). How multilingual is multilingual BERT? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 4996–5001). https://doi.org/10.18653/v1/P19-1493

[27] Raj, M., & Kumar, R. (2026). How much data in low-resource Indian languages is "sufficient" for transfer learning: A comparative study for POS annotation. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 25(1), 1–26. https://doi.org/10.1145/3783981

[28] Ruder, S., Constant, N., Botha, J., Siddhant, A., Firat, O., Fu, J., Liu, P., Hu, J., Garrette, D., Neubig, G., & Johnson, M. (2021). XTREME-R: Towards more challenging and nuanced multilingual evaluation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (pp. 10215–10245). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.emnlp-main.802

[29] Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., & Gurevych, I. (2021). How good is your tokenizer? On the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational*

Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers) (pp. 3118–3135). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.acl-long.243

[30] Shah, R., Chawla, K., Eidnani, D., Shah, A., Du, W., Chava, S., Raman, N., Smiley, C., Chen, J., & Yang, D. (2022). When FLUE meets FLANG: Benchmarks and large pretrained language model for financial domain. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing* (pp. 2322–2335). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.emnlp-main.148

[31] Sundararajan, M., Taly, A., & Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning* (PMLR Vol. 70, pp. 3319–3328). https://proceedings.mlr.press/v70/sundararajan17a.html

[32] Wu, S., & Dredze, M. (2019). Beto, Bentz, Becas: The surprising cross-lingual effectiveness of BERT. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 833–844). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1077

APPENDIX 1

SCHEMA, LABELS AND IDENTIFIERS

A1.1 CORPUS ROW SCHEMA FOR TASK 1

Task 1 uses the canonical columns of Table 3.3 except that the label columns are replaced by the columns in Table A1.1.

Table A1.1: Additional columns for the numeral task

Column	Type	Meaning
number_indic	string	The number as written in the source script
number_english	string	The same value in ASCII digits
start_posn	integer	Character offset into the text, end-exclusive
end_posn	integer	text[start_posn:end_posn] equals number_indic
magnitude	integer	The upstream scale marker
span_recovered	boolean	Null for native rows; whether the numeral was located in the translation

Upstream offsets are exact: the slice reproduces the number on 500 of 500 sampled rows in each language, and on all 22,786 spans as shipped. On a translated row, an unrecovered span carries offsets of minus one and the flag span_not_recovered.

A1.2 LABEL SETS

Task 2 uses two labels, sustainable and unsustainable, shared identically by all three languages. Task 3 uses ten ESG themes, and every language holds the same 532 headlines with the same class counts (Table A1.2). The class sizes are uneven, with the largest class four times the size of the smallest, which is why macro-F1 is the primary metric.

Table A1.2: Task 3 classes and counts in each language

ESG theme	Count	Share
Climate change	92	17.3%
Corporate governance	91	17.1%
Environmental opportunities	72	13.5%
Product liability	68	12.8%
Natural capital	50	9.4%
Pollution waste	44	8.3%

ESG theme	Count	Share
Human capital	37	7.0%
Corporate behavior	30	5.6%
Social opportunities	27	5.1%
Stake holder opposition	21	3.9%

A1.3 IDENTIFIER RULES

An identifier is derived from content, never from row position.

A task 1 identifier includes the span, because a sentence with three numbers forms three items.

Exact duplicate rows receive an occurrence suffix (#1, #2). Task 2 has four such rows, identical in both text and label.

Rows that do not align across languages keep a block-local identifier and the flag unaligned.

Malayalam has no block of its own: it appears only as a translation target within blocks H, B and T.

APPENDIX 2

CONFIGURATIONS

A2.1 TRANSLATION AND VERIFICATION SETTINGS

Table A2.1: Frozen translation decoding configuration

Setting	Value
Model	ai4bharat/indictrans2-indic-indic-1B
Beam size	5
Maximum length	256 tokens
Sampling	None
Key-value cache	Disabled (use_cache = False), a correctness requirement
Device	GPU only; automatic CPU fallback refused
Batch size	Adaptive, excluded from the decoding fingerprint
Language codes	hin_Deva, ben_Beng, tel_Telu, mal_Mlym

Table A2.2: Verification settings

Setting	Value
Similarity model	sentence-transformers/LaBSE, L2-normalised embeddings
Threshold τ	Task 1: 0.82; task 2: 0.82; task 3: 0.73
Audit sample size	200 source sentences per language pair
Entity preservation bar	95% per direction
Truncation rule	Below 0.5 of the direction's own median length ratio
Drift trigger for review	More than 20% of any direction below τ

A2.2 TRAINING AND DIAGNOSTIC SETTINGS

The fine-tuning settings are in Table 4.4. The diagnostic thresholds are in Table A2.3.

Table A2.3: Diagnostic thresholds

Setting	Value
Tokenizer fragmentation threshold	R_frag of 2.0 or more
Saliency divergence threshold	95th percentile on pairs correct in both languages: task 2 IndicBERT 0.038, task 2 mBERT 0.047, task 3 IndicBERT 0.085, task 3 mBERT 0.078
Integrated Gradients convergence limit	Completeness error of 5%
Class collapse	One class predicted on at least 75% of the condition's rows

Setting	Value
ESG lexicon	18 concepts, curated in Hindi and translated with the corpus model
Seeds	Three per configuration, bootstrap of 1,000 samples

APPENDIX 3

PER-DIRECTION QUALITY TABLES

A3.1 ENTITY PRESERVATION

The tables list, for each direction, the share of rows with all entities preserved after excluding corrupted rows, and the counts of rows with dropped entities, altered entities, lost currency symbols, lost percentages and corrupted output.

Table A3.1: Entity preservation by direction, task 1

Direction	Rows	Preserved	Dropped	Altered	Currency lost	Percent lost	Corrupted
ben → mal	5,698	49.8%	6,159	2,469	265	538	432
hin → mal	10,356	56.9%	8,801	2,353	849	2,022	284
ben → tel	5,872	57.8%	4,861	3,116	191	249	258
ben → hin	5,948	58.9%	5,091	2,613	138	34	182
hin → tel	10,442	67.5%	6,564	2,596	503	638	198
tel → mal	5,253	70.8%	2,118	1,282	319	460	763
hin → ben	10,057	70.9%	6,036	2,358	373	122	583
tel → ben	5,194	76.7%	1,869	631	212	101	822
tel → hin	5,322	78.7%	1,628	903	180	92	694

Table A3.2: Entity preservation by direction, task 2

Direction	Rows	Preserved	Dropped	Altered	Currency lost	Percent lost	Corrupted
hin → mal	2,236	89.5%	130	28	103	80	2
ben → mal	2,226	89.8%	340	127	53	74	2
tel → hin	2,072	90.8%	121	24	98	14	0
hin → tel	2,237	91.6%	90	32	92	34	1
ben → tel	2,228	92.0%	289	127	42	39	0
tel → ben	2,067	92.1%	76	23	108	9	5
tel → mal	2,070	93.4%	120	25	29	35	2
ben → hin	2,228	94.0%	313	159	0	3	0
hin → ben	2,231	95.7%	64	29	51	5	7

Table A3.3: Entity preservation by direction, task 3

Direction	Rows	Preserved	Dropped	Altered	Currency lost	Percent lost	Corrupted
hin → mal	531	77.4%	65	52	78	7	1
ben → mal	532	81.0%	79	60	56	4	0
tel → mal	531	86.1%	61	50	19	5	1
hin → tel	532	88.3%	20	13	45	6	0
tel → ben	531	91.5%	34	24	16	4	1
tel → hin	532	91.9%	34	22	14	7	0
ben → tel	532	92.7%	26	12	24	4	0
hin → ben	530	93.6%	14	8	21	6	2
ben → hin	532	97.0%	23	10	1	5	0

A3.2 SIMILARITY DISTRIBUTIONS

Table A3.4: LaBSE similarity distribution by direction, task 1 (τ = 0.82)

Direction	Rows	Mean	Median	5th pct	Below τ	Share
ben → mal	6,130	0.834	0.844	0.720	2,090	34.1%
ben → tel	6,130	0.842	0.849	0.743	1,930	31.5%
ben → hin	6,130	0.846	0.854	0.745	1,801	29.4%
tel → ben	6,016	0.874	0.909	0.631	1,188	19.8%
tel → mal	6,016	0.876	0.906	0.661	1,133	18.8%
tel → hin	6,016	0.870	0.916	0.540	934	15.5%
hin → mal	10,640	0.892	0.906	0.783	1,154	10.9%
hin → ben	10,640	0.917	0.935	0.804	691	6.5%
hin → tel	10,640	0.923	0.937	0.824	498	4.7%

Table A3.5: LaBSE similarity distribution by direction, task 2 (τ = 0.82)

Direction	Rows	Mean	Median	5th pct	Below τ	Share
ben → mal	2,228	0.896	0.902	0.817	121	5.4%
hin → mal	2,238	0.902	0.908	0.824	99	4.4%
ben → tel	2,228	0.916	0.923	0.845	53	2.4%
tel → mal	2,072	0.918	0.924	0.848	33	1.6%
ben → hin	2,228	0.925	0.931	0.860	17	0.8%
tel → ben	2,072	0.933	0.939	0.876	14	0.7%
tel → hin	2,072	0.937	0.943	0.885	8	0.4%
hin → ben	2,238	0.944	0.950	0.895	7	0.3%
hin → tel	2,238	0.939	0.945	0.881	5	0.2%

Table A3.6: LaBSE similarity distribution by direction, task 3 (τ = 0.73)

Direction	Rows	Mean	Median	5th pct	Below τ	Share
hin → mal	532	0.846	0.859	0.737	24	4.5%
tel → mal	532	0.851	0.861	0.746	20	3.8%
ben → mal	532	0.860	0.870	0.762	11	2.1%
tel → ben	532	0.871	0.881	0.775	10	1.9%
tel → hin	532	0.882	0.891	0.786	6	1.1%
ben → tel	532	0.879	0.889	0.777	6	1.1%
hin → tel	532	0.893	0.903	0.801	4	0.8%
ben → hin	532	0.891	0.898	0.798	2	0.4%
hin → ben	532	0.899	0.908	0.811	2	0.4%

APPENDIX 4

FULL TRANSFER GAP TABLES

A4.1 TASK 2 AND TASK 3 TRANSFER GAPS

The tables give the mean transfer gap with its seed standard deviation, the source model's in-language score on the evaluation items (a training-text ceiling), and its score on the target. Targets other than Malayalam are evaluated on native text; Malayalam is machine-translated from a different block.

Table A4.1: Transfer gap by source, target and encoder, task 2

Encoder	Source	Target	Gap (mean ± std)	Source score	Target score
IndicBERT	ben	hin	0.263 ± 0.053	0.956	0.693
IndicBERT	ben	mal	0.367 ± 0.043	0.956	0.589
IndicBERT	ben	tel	0.404 ± 0.015	0.956	0.552
IndicBERT	hin	ben	0.448 ± 0.072	0.954	0.506
IndicBERT	hin	mal	0.387 ± 0.057	0.954	0.567
IndicBERT	hin	tel	0.453 ± 0.038	0.954	0.501
IndicBERT	tel	ben	0.323 ± 0.047	0.957	0.634
IndicBERT	tel	hin	0.361 ± 0.105	0.957	0.596
IndicBERT	tel	mal	0.417 ± 0.005	0.957	0.541
mBERT	ben	hin	0.105 ± 0.023	0.975	0.870
mBERT	ben	mal	0.179 ± 0.028	0.975	0.796
mBERT	ben	tel	0.176 ± 0.010	0.975	0.799
mBERT	hin	ben	0.142 ± 0.010	0.975	0.833
mBERT	hin	mal	0.209 ± 0.007	0.975	0.766
mBERT	hin	tel	0.236 ± 0.056	0.975	0.738
mBERT	tel	ben	0.196 ± 0.058	0.971	0.775
mBERT	tel	hin	0.159 ± 0.060	0.971	0.812
mBERT	tel	mal	0.167 ± 0.056	0.971	0.804

Table A4.2: Transfer gap by source, target and encoder, task 3

Encoder	Source	Target	Gap (mean ± std)	Source score	Target score
IndicBERT	ben	hin	0.523 ± 0.163	0.590	0.067
IndicBERT	ben	mal	0.515 ± 0.115	0.590	0.075
IndicBERT	ben	tel	0.504 ± 0.132	0.590	0.086
IndicBERT	hin	ben	0.510 ± 0.168	0.596	0.086

Encoder	Source	Target	Gap (mean ± std)	Source score	Target score
IndicBERT	hin	mal	0.504 ± 0.184	0.596	0.092
IndicBERT	hin	tel	0.514 ± 0.170	0.596	0.082
IndicBERT	tel	ben	0.478 (1 seed)	0.585	0.108
IndicBERT	tel	hin	0.478 (1 seed)	0.585	0.107
IndicBERT	tel	mal	0.456 (1 seed)	0.585	0.129
mBERT	ben	hin	0.494 ± 0.004	0.810	0.316
mBERT	ben	mal	0.538 ± 0.031	0.810	0.272
mBERT	ben	tel	0.523 ± 0.019	0.810	0.287
mBERT	hin	ben	0.516 ± 0.049	0.805	0.290
mBERT	hin	mal	0.561 ± 0.050	0.805	0.244
mBERT	hin	tel	0.516 ± 0.073	0.805	0.289
mBERT	tel	ben	0.524 ± 0.034	0.784	0.260
mBERT	tel	hin	0.489 ± 0.025	0.784	0.295
mBERT	tel	mal	0.528 ± 0.054	0.784	0.256

For task 3 IndicBERT, Bengali-source cells use two converged seeds and Telugu-source cells use one, because seeds that never trained are excluded from the source comparison. A cell with one seed has no standard deviation.

APPENDIX 5

REPRODUCTION AND SCRIPT INVENTORY

A5.1 SETTING UP

The environment is a Python 3.11 virtual environment with the pinned requirements installed. The IndicTrans2 and IndicBERT repositories are gated on the model hub, so the licence must be accepted and the machine authenticated before the first download. Every command is run from the repository root as a module.

A5.2 COMMANDS

Table A5.1: Principal commands by phase

Step	Command
Fetch upstream data	python -m src.download_dataset.download
Independence audit	python -m scripts.t102_audit --task N --independence
Align parallel splits (tasks 2, 3)	python -m scripts.t102b_align --task N
Ingest native splits	python -m scripts.t103_ingest --task N
Generate translations	python -m scripts.t106_generate --task N
Rebuild splits from checkpoints	python -m scripts.t106_generate --task N --rebuild
Entity preservation	python -m scripts.t105_entities --task N
Integrity checks	python -m scripts.t107_integrity --task N --write-flags
LaBSE scoring	python -m scripts.t108_labse --task N
Drift report	python -m scripts.t109_drift --task N --write-flags
Calibrate τ	python -m scripts.t110_calibrate --task N
Direction ranking	python -m scripts.t111_ranking --task N
Freeze and verify	python -m scripts.t112_freeze --task N [--verify]
Condition matrix	python -m scripts.t113_conditions --task N
Datasheet	python -m scripts.t114_datasheet [--check]
Baselines and gate	python -m scripts.t206_baseline; python -m scripts.t207_gate
Transfer runs	python -m scripts.t305_run_conditions
Source and encoder comparison	python -m scripts.t408_source_comparison; python -m scripts.t504_comparison
Diagnostics	python -m scripts.t601_diagnostics
Findings	python -m scripts.t707_findings
Dashboard	streamlit run dashboard/app.py

A5.3 PHASE AND TASK INVENTORY

Table A5.2: Corpus tasks and what each delivers

Task	Script	Delivers
T-102	t102_audit	Native-split audit and independence verdict per task
T-102b	t102b_align	Item correspondence for tasks 2 and 3
T-103	t103_ingest	Identifier scheme, label schema, input/output layer
T-104	t104_smoke	Translation pipeline, frozen decoding, model choice
T-105	t105_entities	Numeral and entity preservation checker
T-106	t106_generate	All nine directions for each task
T-107	t107_integrity	Structural and corruption checks
T-108	t108_labse	Similarity of every machine-translated pair
T-109	t109_drift	Per-direction drift report
T-110	t110_calibrate	Per-task threshold calibration
T-111	t111_ranking	Direction ranking
T-112	t112_freeze	Frozen, hashed release
T-113	t113_conditions	Evaluation-condition matrix and validator
T-114	t114_datasheet	Generated datasheet

The datasheet and the generated reports under the reports directory hold every figure quoted in this document. The source code and the released corpus are available at https://github.com/souparno-nag/Cross-Linguistic_Transfer_Diagnotic_for_Financial_NLP_in_Indic_Languages.